pyplant

A two-tank process plant with its control, SCADA and fault-diagnosis stack, in Python

User Manual and Technical Reference

pyplant project

Version 0.1.0 – October 2026

Contents

1 Introduction **1**
- 1.1 What pyplant is . . . 1
- 1.2 Who this manual is for . . . 1
- 1.3 The process in one picture . . . 2
- 1.4 Architecture . . . 2
- 1.5 The two execution paths . . . 3
- 1.6 Conventions in this manual . . . 3

2 Installation and first run **4**
- 2.1 Requirements . . . 4
- 2.2 Installing . . . 4
- 2.3 Checking the installation . . . 5
- 2.4 Starting the workbench . . . 5
- 2.5 The workspace directory . . . 5
- 2.6 Running the stack by hand . . . 6

3 The workbench **7**
- 3.1 Layout . . . 7
- 3.2 Quick start tab . . . 8
- 3.3 Runtime tab . . . 9
- 3.4 Faults tab . . . 10
- 3.5 Data tab . . . 10
- 3.6 Model tab . . . 10
- 3.7 Speed . . . 11

4 Walkthrough: one leak, from step 0 to a trained detector **12**
- 4.1 Step 0: open the workbench . . . 12
- 4.2 Step 1: run the default scenario offline . . . 13
- 4.3 Step 2: look at the Runtime tab before starting . . . 14
- 4.4 Step 3: start the stack . . . 15
- 4.5 Step 4: read the mimic . . . 16
- 4.6 Step 5: operate LIC101, a setpoint step . . . 16
- 4.7 Step 6: open the Faults tab . . . 17
- 4.8 Step 7: inject the leak . . . 18
- 4.9 Step 8: check the ground truth . . . 19
- 4.10 Step 9: watch the controller compensate . . . 20
- 4.11 Step 10: see it as the operator, on the HMI . . . 20
- 4.12 Step 11: clear the fault and open the Data tab . . . 21
- 4.13 Step 12: generate a Quick dataset . . . 22
- 4.14 Step 13: the dataset is ready . . . 23
- 4.15 Step 14: set up training . . . 24

4.16 Step 15: train and read the results . . . 24
4.17 Step 16: check the leak, fault mode by fault mode . . . 25
4.18 Step 17: stop the stack . . . 26
4.19 Numbers to expect . . . 27

5 Using the toolbox step by step **28**
5.1 Tutorial 1: the default scenario . . . 28
5.2 Tutorial 2: the live plant . . . 29
5.3 Tutorial 3: faults on the live plant . . . 29
5.4 Tutorial 4: a dataset . . . 29
5.5 Tutorial 5: training and comparing detectors . . . 30
5.6 Tutorial 6: the same pipeline from Python . . . 30

6 The plant model **31**
6.1 State, inputs and outputs . . . 31
6.2 Mass balances . . . 31
6.3 Gravity drains: Torricelli's law . . . 32
6.4 Pump and valve: an algebraic loop solved in closed form . . . 32
6.5 Valve actuator . . . 32
6.6 Numerical integration . . . 33
6.7 Steady state . . . 33
6.8 Sizing and the design point . . . 33
6.9 Linearisation and the process time constant . . . 33
6.10 The simulation stepper . . . 34

7 Control theory and the LIC101 loop **35**
7.1 Feedback control . . . 35
7.2 The PID law . . . 35
7.3 IMC tuning . . . 36
7.4 Sampling and the zero-order hold . . . 37
7.5 Why integral action hides a leak . . . 37
7.6 Controller parameters . . . 38

8 The soft PLC **39**
8.1 The scan cycle . . . 39
8.2 The program . . . 40
8.3 Drift-free scheduling . . . 41
8.4 The tag dictionary . . . 42
8.5 Modbus TCP: the field layer . . . 42
8.6 OPC UA: the information layer . . . 43

9 SCADA: historian, HMI and alarms **44**
9.1 The historian . . . 44
9.2 The operator HMI . . . 45
9.3 Alarm management (ISA-18.2) . . . 46

10 Faults and scenarios **48**
10.1 Where a fault enters . . . 48
10.2 Timing . . . 49
10.3 The fault catalogue . . . 49
10.4 Valve stiction in detail . . . 49
10.5 Instrument noise is not a fault . . . 49
10.6 Scenarios . . . 50

10.7 Injecting faults into the live stack . . . 51

11 Machine learning for fault detection **52**
11.1 The pipeline . . . 52
11.2 Why unsupervised, trained on normal only . . . 52
11.3 Windows . . . 52
11.4 Features . . . 53
11.5 Detector 1: z-score limits . . . 54
11.6 Detector 2: PCA and the SPE statistic . . . 54
11.7 Detector 3: isolation forest . . . 55
11.8 Thresholds . . . 55
11.9 The health index . . . 55
11.10 Isolation: contributions . . . 56
11.11 Using the detectors from Python . . . 56

12 Deep learning: the autoencoder detector **58**
12.1 Neural network fundamentals . . . 58
12.2 The autoencoder . . . 59
12.3 Scoring, threshold and isolation . . . 60
12.4 Practical advice . . . 60

13 Evaluating and selecting a detector **61**
13.1 Split by scenario, never by sample . . . 61
13.2 Persistence . . . 61
13.3 The metrics . . . 62
13.4 Selecting the detector . . . 62
13.5 Reference results . . . 63
13.6 Reading the Model tab . . . 63

14 Changing parameters **65**
14.1 Where a parameter takes effect . . . 65
14.2 The workspace configuration . . . 65
14.3 Command-line options . . . 67
14.4 Code constants . . . 68
14.5 Offline experiments with other parameters . . . 69

15 Extending the toolbox **70**
15.1 Adding a tag . . . 70
15.2 Adding a fault . . . 70
15.3 Adding a detector . . . 71
15.4 Adding an online inference service . . . 72
15.5 Testing . . . 72

16 API reference **74**
16.1 Plant control API (port 5401) . . . 74
16.2 Historian query API (port 5404) . . . 75
16.3 HMI API (port 5405) . . . 75
16.4 Industrial protocols . . . 75
16.5 Python API . . . 75

17 Troubleshooting **77**
17.1 Installation and start-up 77
17.2 The live stack 78
17.3 Data and models 78
17.4 Starting over 79

A Reference **80**
A.1 Tag list 80
A.2 File layout 81
A.3 Glossary 81

Chapter 1

Introduction

1.1 What pyplant is

pyplant is a complete, simulated industrial control system for a small process plant, written entirely in Python. It contains every layer a real installation has, from the physics at the bottom to a data-driven fault diagnosis service at the top:

- a *physics simulator* of two gravity-drained tanks fed by a centrifugal pump through a control valve;
- a *soft PLC* with a deterministic scan cycle, a velocity-form PID controller, alarms and a latched pump interlock;
- a *field network* using Modbus TCP between the plant and the PLC;
- an *OPC UA server* that publishes every signal with engineering units and ranges;
- a *historian* that stores samples in DuckDB and answers step-hold window queries;
- an *operator HMI* with a mimic diagram, trends and an ISA-18.2 alarm list;
- a *fault injection layer* and a *scenario runner* that produce perfectly labelled datasets;
- a *machine-learning toolbox* with four anomaly detectors, from z-score limits to a neural autoencoder, evaluated with operator-facing metrics;
- the *workbench*: one web page that drives all of the above.

Nothing requires hardware or vendor software. Every number in this manual can be reproduced on a laptop.

1.2 Who this manual is for

The manual serves three audiences, and each chapter says which one it is written for.

Users who want to run the plant, break it and train a detector. Read Chapters 2, 3 and 5.

Students of control, automation or machine learning who want the theory behind each layer. Chapters 6 to 13 derive every model and algorithm the code implements.

Developers who want to change parameters, add a tag, a fault or a detection method. Chapters 14 to 16.

1.3 The process in one picture

Figure 1.1 is the piping and instrumentation view of the plant. A pump (PMP101) delivers water through an equal-percentage control valve (FCV101) into tank 1. Tank 1 drains by gravity through an orifice into tank 2, which drains to atmosphere. The level controller LIC101 holds the level in tank 1 (LT101) at its setpoint by moving the valve. Tank 2 is not controlled, so disturbances in tank 1 reach it with a lag, which is useful for diagnosis.

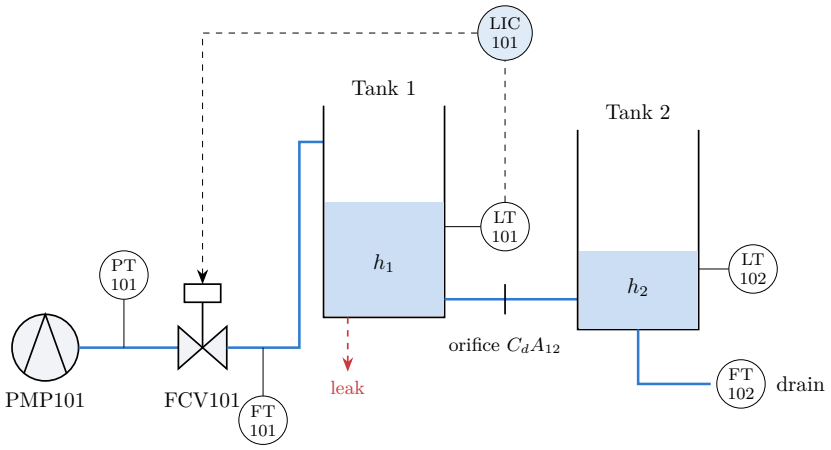


Figure 1.1: Process and instrumentation of the two-tank plant. Solid blue lines are piping, dashed black lines are signals. The red dashed arrow is where the leak fault opens a hole in tank 1.

1.4 Architecture

The software is split into processes exactly as a real plant is split into devices. Figure 1.2 shows the four runtime services, the networks between them, and the workbench that supervises them.

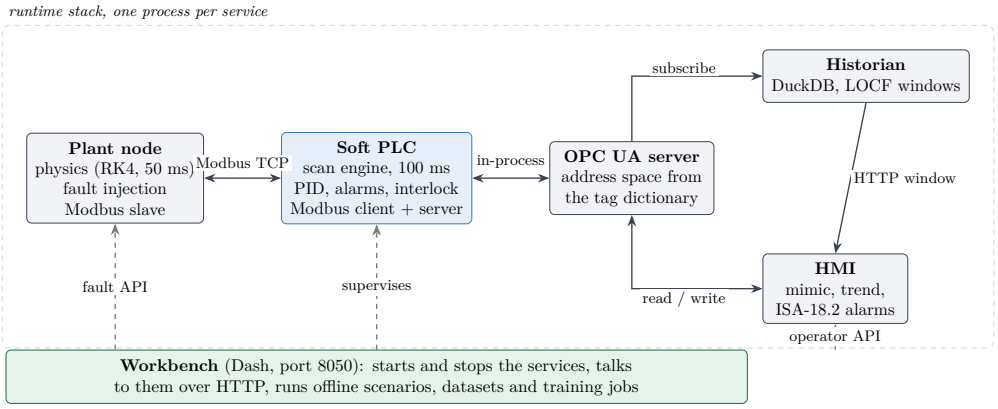


Figure 1.2: Runtime architecture. Each box in the dashed frame is a separate operating-system process. The workbench is a client of the stack, never a participant in it.

Three design rules recur throughout the code base and this manual:

1. **One source of truth per fact.** Every signal is declared once, in the tag dictionary `pyplant/tags.py`. Addresses, scale factors, alarm limits and OPC UA paths are derived from it, never retyped.
2. **Ownership.** Every tag has exactly one owner process. A process writes only what it owns. The plant cannot write controller outputs and the HMI cannot write measurements.

3. **The same code drives the live plant and the datasets.** The offline scenario runner and the live plant node step the identical `Simulation` object with the identical PLC `Program`, so the training data cannot drift away from the running plant.

1.5 The two execution paths

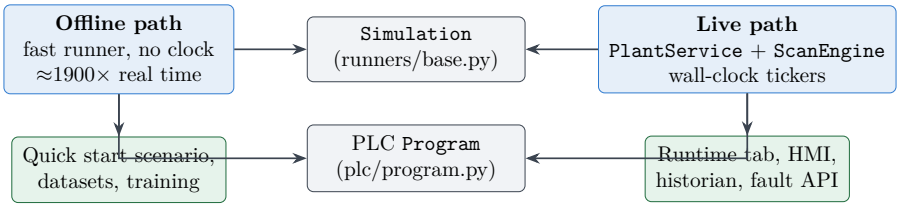


Figure 1.3: Two drivers, one model. Neither path owns the physics or the logic.

The *offline path* (Figure 1.3, left) runs the plant and the PLC program in a tight loop with no clock and no network. A simulated hour takes under two seconds. It is used by the Quick start tab, by dataset generation and by the tests. The *live path* (right) runs the same objects against wall-clock schedulers, splits them into processes and connects them with Modbus and OPC UA. It is used whenever the stack is started.

1.6 Conventions in this manual

- Tag names are set in sans serif, e.g. LT101.
- File names, Python identifiers and commands are set in typewriter type, e.g. `pyplant/plc/pid.py`.
- Buttons and labels in the workbench are set in bold, e.g. **Start stack**.
- Physical quantities inside the code are SI. Engineering units (mm, L/min, kPa, %) appear only at the instrument boundary.

Chapter 2

Installation and first run

2.1 Requirements

Item	Requirement
Python	3.11 or newer (3.12 is used for development)
Operating system	Windows, Linux or macOS
Memory	2 GB free is ample
Runtime libraries	numpy, pandas, pymodbus 3.8.x, asyncua, duckdb, fastapi, uvicorn, httpx
Machine learning (`ml` extra)	scikit-learn, pyarrow, joblib
Workbench UI (`ui` extra)	dash, plotly
Tests (`dev` extra)	pytest

pymodbus must stay on 3.8.x

The 3.15 line of pymodbus replaced the datastore API this project uses and no longer offers a way to update served values from outside the server, which is what a live plant does twenty times a second. `pyproject.toml` pins `pymodbus>=3.8,<3.9`; do not relax it.

2.2 Installing

From the directory that contains `pyproject.toml`:

```
# a dedicated environment is recommended
conda create -n pyplant python=3.12 -y
conda activate pyplant

pip uninstall -y pyplant # an unrelated PyPI package has this name
pip install -e ".[ml,ui,dev]" # everything: stack, ML, workbench, tests
python -c "import pyplant.hist.db; print('ok')"
```

The `-e` (editable) flag puts this directory on the Python path, so an edited file takes effect without reinstalling and `python -m pyplant...` works from any directory.

Windows with several Pythons

If `python -version` reports something older than 3.11, call the environment's interpreter explicitly, for example

```
C:\Users\you\anaconda3\envs\pyplant\python.exe -m pyplant.workbench.app
```

2.3 Checking the installation

```
python -m pyplant.tags # prints the tag list and the Modbus block plan
pytest -q # full suite, about five minutes
pytest -q -m "not slow" # skips tests that start real subprocesses
```

2.4 Starting the workbench

```
python -m pyplant.workbench.app --workspace ./work/demo --base-port 5400
```

Then open `http://127.0.0.1:8050`. The options are:

Option	Default	Meaning
`-workspace`	`./work/demo`	directory holding config, data, models and logs
`-base-port`	5400	first of six consecutive ports for the stack
`-host`	127.0.0.1	interface the workbench page listens on
`-port`	8050	port of the workbench page
`-debug`	off	Dash debug mode with hot reload

With `-base-port 5400` the stack uses the ports in Table 2.1. Several people on one machine can each run a stack by choosing different base ports.

Port	Offset	Service	Protocol
5400	+0	plant Modbus slave	Modbus TCP
5401	+1	plant fault-injection API	HTTP
5402	+2	PLC Modbus server (SCADA side)	Modbus TCP
5403	+3	PLC OPC UA endpoint, `/pyplant/`	OPC UA
5404	+4	historian query API	HTTP
5405	+5	HMI operator page and API	HTTP

Table 2.1: Ports allocated from the base port.

2.5 The workspace directory

Everything a session produces lives in the workspace, and nothing it produces lives anywhere else:

```
work/demo/
  config.json speed, dt, ports, plant, loop and data settings
  runs.jsonl append-only log of every action and its result
  dataset.parquet the generated, labelled dataset
```

```
history.duckdb the historian database of the live stack
artifacts/ trained detectors, manifest.json, metrics.json, per-run CSVs
jobs/ progress files of background jobs and service logs
```

Deleting the workspace is always safe: the next start recreates it with default settings.

2.6 Running the stack by hand

The workbench starts the four services for you, but each can be started from a terminal. This is useful for debugging and is exactly what the workbench runs (the Runtime tab shows the command lines).

```
python -m pyplant.services.plant --port 5020 --control-port 5021
python -m pyplant.services.plc --plant-port 5020 --port 5022 --opcua-port 5023
python -m pyplant.services.historian --endpoint opc.tcp://127.0.0.1:5023/pyplant/ \
                                  --db history.duckdb --api-port 5024
python -m pyplant.services.hmi --endpoint opc.tcp://127.0.0.1:5023/pyplant/ \
                                  --history http://127.0.0.1:5024 --port 5025
```

Start them in this order; each depends on the one before it. All command-line options are listed in Section 14.3.

Chapter 3

The workbench

The workbench is the single web page from which the whole toolbox is driven. It holds no process state of its own: every value on the screen was read from the HMI, the plant's fault API or the historian a moment ago. Long jobs (dataset generation, training) run in separate processes and report progress through files, so the page never blocks.

3.1 Layout

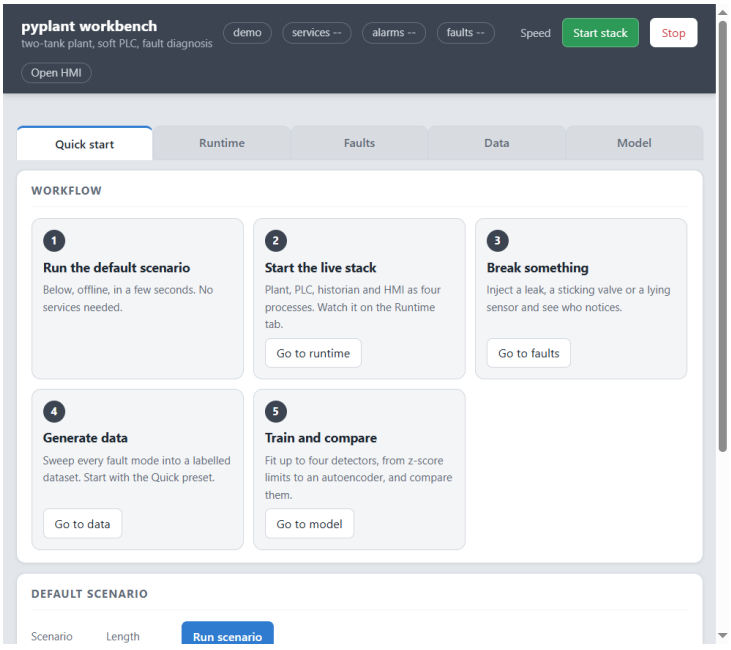


Figure 3.1: The workbench header and the Quick start tab with its five workflow cards.

Header

The dark grey header stays at the top of the page and contains:

- the workspace name;
- three status pills: services up (e.g. **4/4 services**), unacknowledged alarms, and active faults. Green means healthy, amber partial, red attention;
- the **Speed** selector, a real-time factor applied when the stack starts (Section 3.7);
- **Start stack** and **Stop**, which launch or terminate the four services in dependency order;

- **Open HMI**, a link to the stand-alone operator page.

Tabs

Below the header are five tabs. All of them are present in the page at all times and merely hidden, so switching is instant and no callback ever fires against a missing component.

Tab	Purpose
Quick start	the opening page: workflow cards, the default scenario run offline, and a one-click live demo
Runtime	service status, live mimic diagram, LIC101 faceplate and operator commands, trend, alarm list
Faults	one-click fault presets, a custom fault builder, the active-fault list, ground truth against instrument readings
Data	generation of the labelled scenario dataset, with presets and progress
Model	selection and training of detection methods, comparison table, per-fault-mode results

3.2 Quick start tab

3.2.1 Workflow cards

Five numbered cards describe the recommended order of work. A card turns green when its step is done, judged from what the workspace actually holds:

1. **Run the default scenario** — done once a scenario has run in this session;
2. **Start the live stack** — done while the HMI answers;
3. **Break something** — done once any fault has been injected;
4. **Generate data** — done once `dataset.parquet` exists;
5. **Train and compare** — done once `artifacts/metrics.json` exists.

Each card from step 2 on has a **Go to ...** button that opens the right tab.

3.2.2 Default scenario

When the page opens it immediately runs the default scenario, an abrupt leak in tank 1, through the offline runner. No services are needed and the run takes a few seconds. The controls are:

Scenario any of the nine library scenarios: normal operation, abrupt leak (default), slow leak, pump wear, fouling, valve stiction, sensor bias, sensor drift, frozen sensor.

Length 15, 30 or 60 minutes of plant time. The fault starts at 40% of the length.

setpoint moves off by default, so the setpoint is held at 300 mm and everything that moves is caused by the fault. Ticking it adds the standard setpoint programme (300, 340, 280, 300 mm at 0, 15, 30 and 45 minutes).

Run scenario reruns with the chosen settings.

The summary cards compare the last five minutes before onset with the last five minutes of the run: change in LT101, controller output, LT102 and FT101, the time of any pump trip, and the onset time. The chart has levels on the left axis and percentages on the right; the faulty period is shaded red.

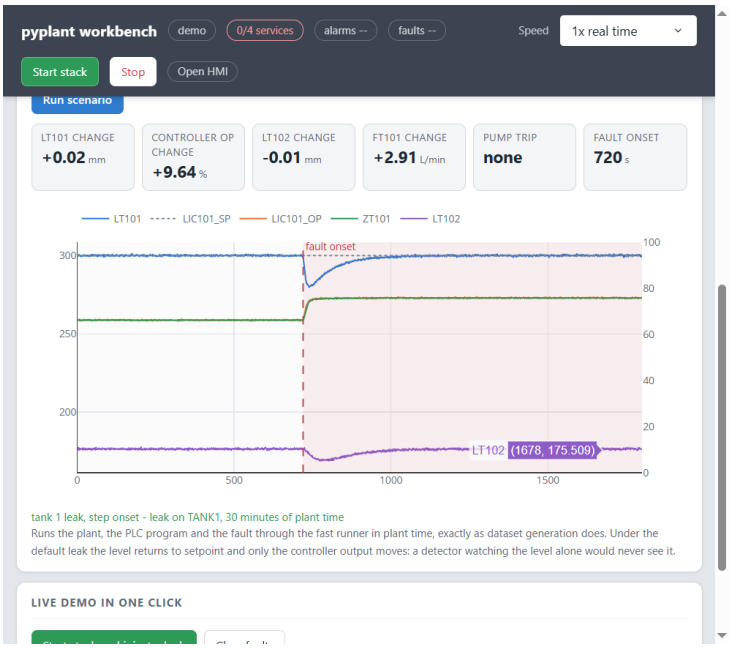


Figure 3.2: The default scenario: an abrupt leak in tank 1 at $t = 720\,\mathrm{s}$. The level (blue) dips and returns to the setpoint (dotted); the controller output (orange) and the valve position (green) rise by about 10%. The summary cards quantify the change.

3.2.3 Live demo in one click

Start stack and inject a leak starts the four services if they are not already running, waits until each answers, and injects a $5 \times 10^{-5}\,\mathrm{m}^2$ leak into the live plant. Open the Runtime tab to watch the controller output climb while the level holds. **Clear faults** removes every injected fault and leaves the instrument noise in place.

3.3 Runtime tab

Services one row per service with state, process id, port and role, and the exact command line the supervisor ran.

Unit 1 the mimic: pump, valve filled to its measured stem position, both tanks filled to their levels, flows and pump pressure. Pipes turn blue when they carry flow. A red frame marks a level alarm and a red banner a tripped interlock. Below it, cards for FT101, FT102, PT101, ZT101, scan time and overrun count.

LIC101 the controller faceplate: PV, SP, OP, error, mode and pump state, plus the operator commands:

- **Setpoint + Write** — 0 to 600 mm. Values outside the span are rejected with a visible error, never clamped.
- **Manual OP + Write** — valve demand in manual mode, 0 to 100%.
- **Auto / Manual** — controller mode (bumpless both ways).
- **Start / Stop** — pump request.
- **Reset interlock** — pulses the reset bit. A latched trip clears only if its cause has gone.

Trend LT101, LIC101_SP and LT102 in mm on the left axis and LIC101_OP in % on the right, over 2, 5, 15 or 60 minutes, read from the historian.

Alarms the ISA-18.2 alarm list with priority, tag, message, state, time in and count, and per-row **Ack** and **Shelve** (30 minutes). **Acknowledge all** acknowledges every alarm.

3.4 Faults tab

Break something seven presets, one click each: leak in tank 1, slow leak (15 minute ramp), sticking valve, worn pump, fouled interconnect, LT101 reading 25 mm high, LT101 frozen. Some print a suggested next step; for instance a frozen transmitter on setpoint gives the controller nothing to act on until you move the setpoint.

Or build one any fault kind with its own magnitude (units shown beside the field), sensor tag for sensor faults, and ramp time. Default magnitudes are filled in when the kind changes.

Active faults id, kind, target, state, magnitude, start and ramp.

Ground truth what each instrument reads against what the plant really is. Under a sensor fault the two disagree; under a process fault they agree and the fault appears in the controller output.

Injection is separate from the control system

Faults are injected through the plant's own API, not through the PLC or the HMI. Neither the operator page nor a detector is told that a fault exists; the only way to know is to observe the plant. This keeps the diagnosis problem honest.

3.5 Data tab

Preset fills the four fields below:

Preset	Run length	Magnitudes	Repeats	Normal runs	Typical time
Quick	1200 s	1	1	6	about 1.5 min
Standard	3600 s	3	2	10	about 4 min
Thorough	3600 s	5	3	16	about 10 min

Run length seconds of recorded plant time per run.

Magnitudes how many fault sizes per mode, spread from 0.5 to 1.5 times the library value.

Repeats runs per mode and magnitude, each with its own seed and a random onset between 25% and 60% of the run.

Normal runs fault-free runs. At least three are needed (train, calibrate, test); more give a better false-alarm estimate.

Generate / Cancel start or stop the background job. The progress bar and **Recent runs** table update every three seconds.

3.6 Model tab

Methods tick any of the four detectors (Chapter 11 and 12): z-score limits, PCA/MSPC, isolation forest, autoencoder.

Window samples per window (1 Hz, so 60 is one minute).

Stride samples between consecutive windows.

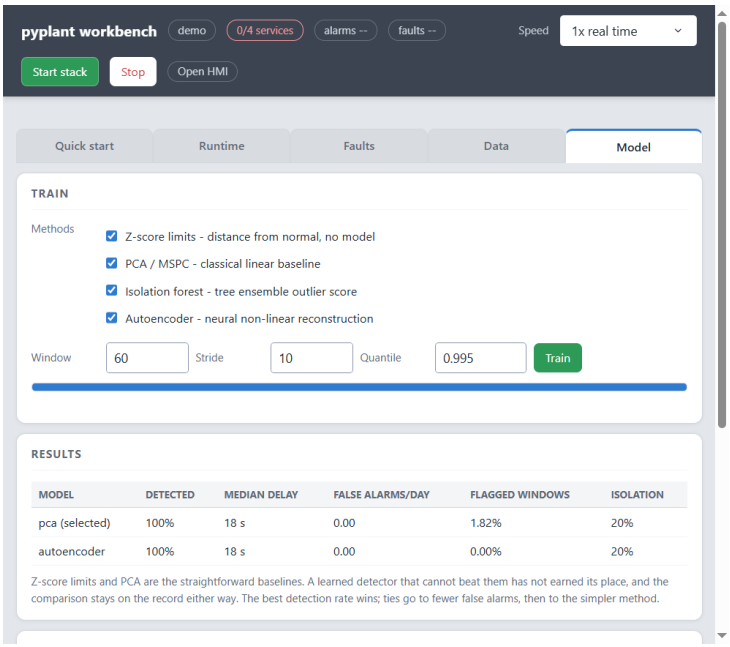


Figure 3.3: The Model tab: method selection, window settings and the comparison table.

Quantile the fraction of held-out normal windows that must lie below the alarm threshold, e.g. 0.995.

Train fits every ticked method, evaluates it and deploys the best.

Results detection rate, median detection delay, false alarms per day, flagged-window rate and isolation accuracy per method.

By fault mode for the selected method: runs, detection rate, median delay, the channel that contributed most, and the channel physics expects.

3.7 Speed

Speed is a real-time factor applied to both the plant and the PLC tickers: at 4× a 100 ms scan runs every 25 ms of wall time. Slow faults need it: a stiction limit cycle takes several process time constants to establish, and this plant's time constant is 72 s.

Windows timer granularity

Windows schedules sleeps in steps of about 15.6 ms. Above 4× the scan period falls below that, every scan lands late and SCAN_OVERRUNS climbs. The controller still works; the diagnostics are telling the truth. Keep speed at 4 or below on Windows. Linux and macOS manage 20×.

Chapter 4

Walkthrough: one leak, from step 0 to a trained detector

This chapter follows a single, fixed scenario through every part of the workbench. Every figure is a real screenshot taken while writing this manual, in the order the steps happen, and each one is followed by a table that names the elements on screen and says what they do. Section 4.19 collects the numbers to expect at each step.

The scenario

Unit 1 runs in automatic at a level of 300 mm. The operator raises the setpoint to 340 mm. Then a hole opens in tank 1 (the **Leak in tank 1** preset, an effective leak area of $5 \times 10^{-5}\,\mathrm{m}^2$). We watch the controller hide the leak, then confirm it on the operator HMI. Finally we generate a labelled dataset and train four detectors to see which one would have caught it. Workspace `work/manual`, base port 5400, speed 1× real time.

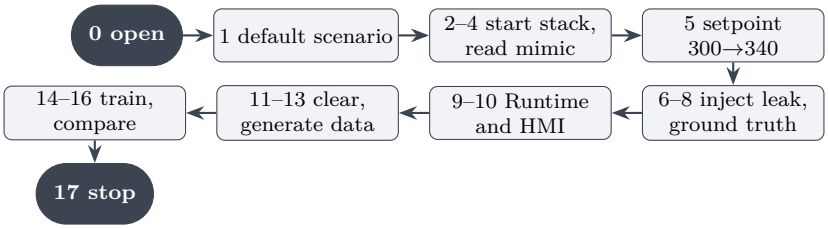


Figure 4.1: The eighteen steps of the walkthrough.

4.1 Step 0: open the workbench

Start the workbench on its own workspace so the walkthrough does not disturb other work:

```
conda activate pyplant
python -m pyplant.workbench.app --workspace ./work/manual
```

Open `http://127.0.0.1:8050`. The Quick start tab opens first (Figure 4.2).

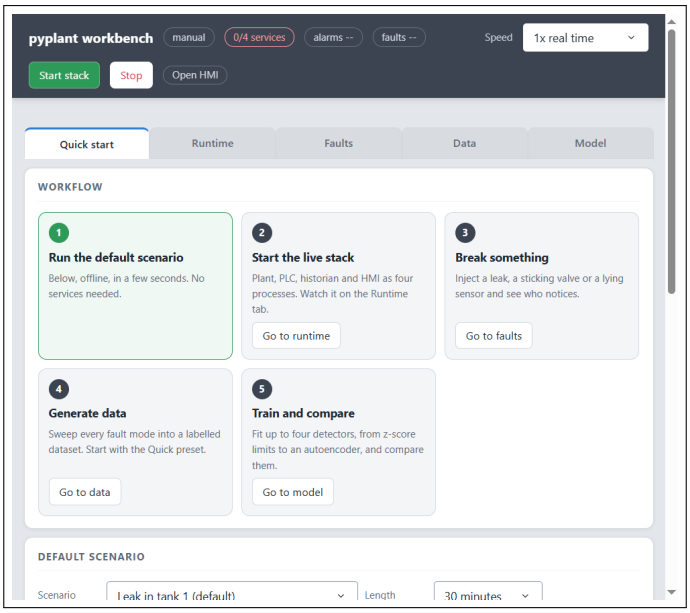


Figure 4.2: Step 0: the workbench as it opens, with nothing running.

Element	*What it shows or does*
pyplant workbench	Title. The dark header stays on screen on every tab.
manual (badge)	Name of the workspace folder. Every file the session writes goes there.
0/4 services (badge)	How many of the four services (plant, PLC, historian, HMI) are up. It is red below 4/4 and green at 4/4.
alarms – **/ faults** –	Live alarm and fault counts. They show "–" while the stack is down, because nobody is there to ask.
Speed	Plant-time multiplier for the live stack (1× to 20×). It takes effect at the next **Start stack**; see Section 3.7.
Start stack	Green button. Launches the four services as separate processes.
Stop	Stops every service the workbench started.
Open HMI	Link that opens the operator page on port 5405 in a new tab.
Tabs	**Quick start**, **Runtime**, **Faults**, **Data**, **Model**. The active tab is white with a blue top edge.
Workflow cards 1–5	The recommended order of work. Card 1 is highlighted green because it needs no services. The **Go to ...** buttons switch tabs.

4.2 Step 1: run the default scenario offline

Scroll down to **Default scenario**. Leave **Scenario** on **Leak in tank 1 (default)**, **Length** on **30 minutes**, and **setpoint moves** unticked, then press **Run scenario**. The fast runner simulates 30 minutes of plant time in a few seconds, without any service.

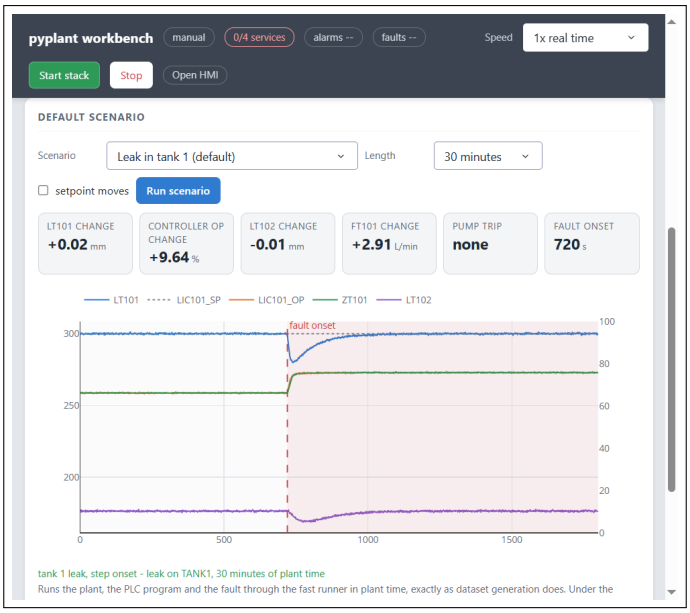


Figure 4.3: Step 1: the default scenario result.

Element	*What it shows or does*
Scenario (dropdown)	Which built-in scenario to run. The default is the tank-1 leak.
Length (dropdown)	Plant time to simulate.
setpoint moves	When ticked, the scenario also steps the setpoint, so the fault has to be told apart from normal operation.
Run scenario	Blue button. Runs the scenario and redraws everything below it.
LT101 CHANGE	End level minus start level: **+0.02 mm**. The loop holds the level, so the leak is *invisible* in the level.
CONTROLLER OP CHANGE	**+9.64 %**. The controller opens the valve to feed the leak. This is where the fault actually shows.
LT102 / FT101 CHANGE	Tank 2 is unaffected (**−0.01 mm**); the inflow rises by **2.91 L/min**, which equals the leak flow.
PUMP TRIP	Whether the low-level interlock tripped (**none**).
FAULT ONSET	Plant time at which the leak opened (**720 s**).
Chart	Left axis: levels (LT101 blue, LT102 purple, setpoint grey dashes). Right axis: LIC101_OP orange and ZT101 green (%). The red band marks the fault.
Caption line	Green text naming the scenario and length that produced the chart.

The chart is the whole lesson of the chapter in one picture. At onset the level dips about 20 mm and recovers within a few minutes. The valve then stays about 10 points more open for as long as the leak lasts.

4.3 Step 2: look at the Runtime tab before starting

Click the **Runtime** tab. With nothing running, every service is down and the mimic shows dashes (Figure 4.4).

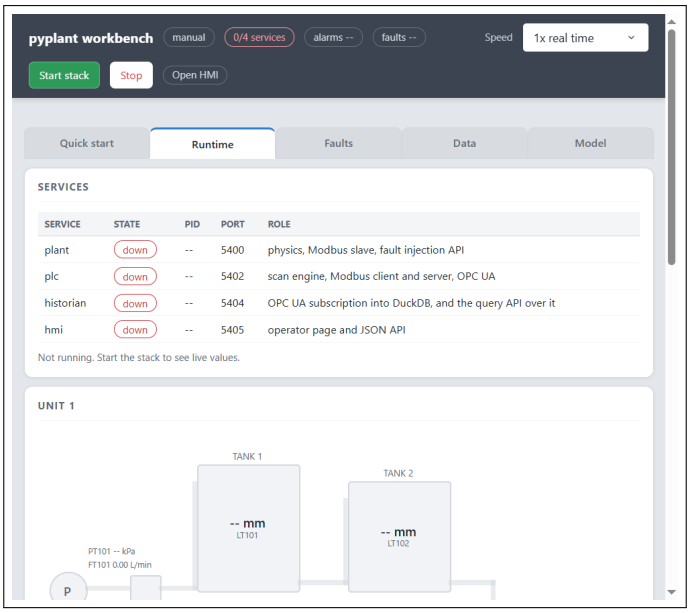


Figure 4.4: Step 2: the Runtime tab with the stack down.

Element	*What it shows or does*
SERVICES table	One row per service: name, **STATE** pill (red **down** / green **up**), operating-system **PID**, **PORT** and **ROLE**.
Hint line	"Not running. Start the stack to see live values."
UNIT 1 mimic	The process drawing. Values are "–" until the PLC answers.

4.4 Step 3: start the stack

Keep **Speed** at **1x real time** and press **Start stack**. After a few seconds the badge turns green at **4/4 services**, and the alarm and fault counts change from "–" to 0.

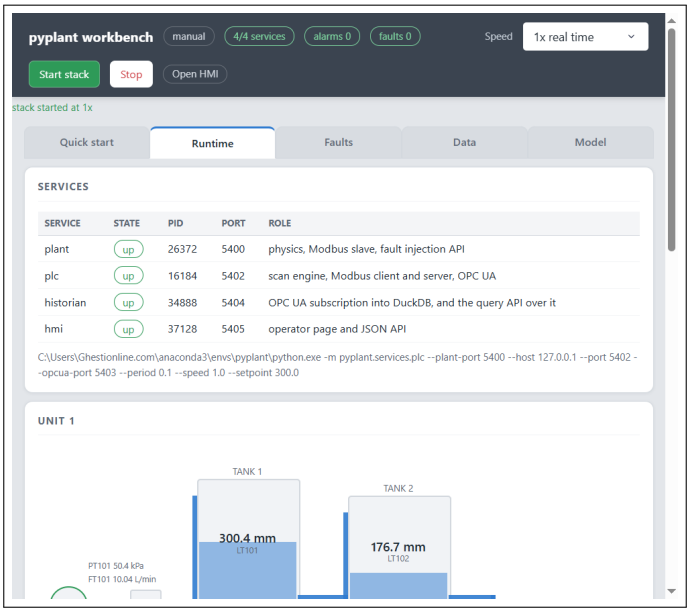


Figure 4.5: Step 3: all four services up.

Element	*What it shows or does*
stack started at 1x	Green confirmation under the header.
STATE = up	Each process answered its health check.
PID	Process id; useful if you need to look at a service in Task Manager.
PORT	plant 5400 (Modbus), plc 5402 (Modbus server; OPC UA on 5403), historian 5404, hmi 5405.
Command line (grey text)	The exact command the supervisor ran for the last service listed. Copy it to run that service by hand.

4.5 Step 4: read the mimic

Scroll to **UNIT 1**. This is the healthy reference picture; keep it in mind for step 9.

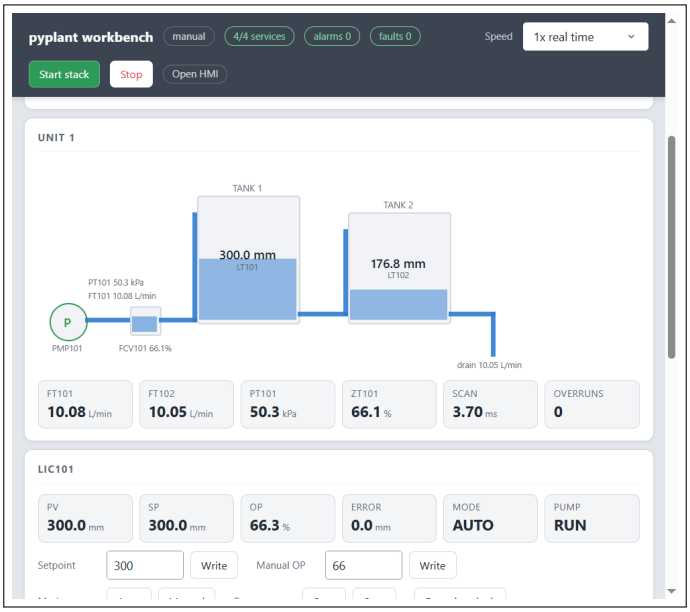


Figure 4.6: Step 4: the healthy plant at steady state.

Element	*What it shows or does*
PMP101	Pump symbol. A green ring means running.
FCV101 66.1%	Control valve. The fill of the small box is the stem position.
PT101 / FT101 labels	Pump discharge pressure (50.3 kPa) and inflow (10.08 L/min), drawn on the pipe.
TANK 1 / TANK 2	Live levels LT101 = 300.0 mm and LT102 = 176.8 mm. The blue fill is to scale.
drain 10.05 L/min	Outflow of tank 2 (FT102). In steady state it equals FT101.
Tiles FT101 ... ZT101	The same measurements as numbers.
SCAN / OVERRUNS	PLC scan time (3.70 ms of a 100 ms budget) and the count of scans that ran over. A non-zero OVERRUNS means the PC is too slow for the chosen speed.

4.6 Step 5: operate LIC101, a setpoint step

On the **LIC101** faceplate, type 340 in **Setpoint** and press its **Write** button. Watch the trend for about two minutes.

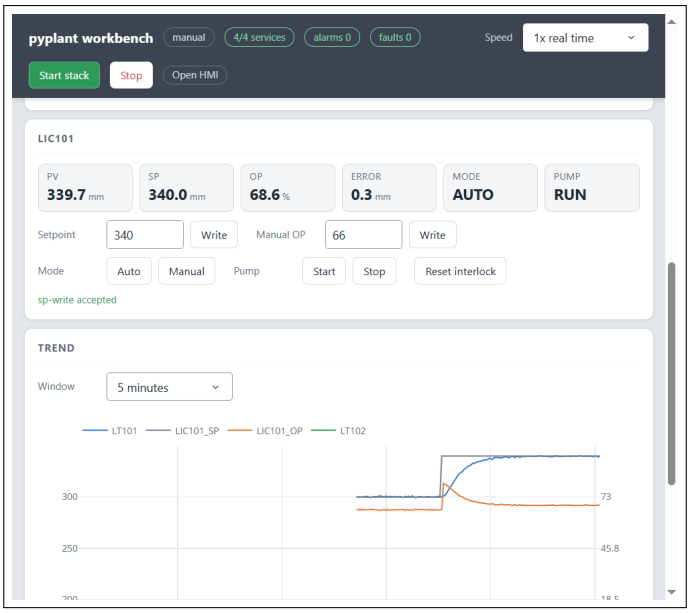


Figure 4.7: Step 5: the faceplate after the setpoint step to 340 mm, with the trend.

Element	*What it shows or does*
PV / SP / OP / ERROR	Process value 339.7 mm, setpoint 340.0 mm, controller output 68.6 %, error 0.3 mm.
MODE / PUMP	AUTO (the PID drives the valve) and RUN.
Setpoint + Write	Sends a new setpoint to the PLC. “sp-write accepted” confirms it.
Manual OP + Write	Valve opening to use in Manual mode. Ignored in Auto.
Auto / Manual	Switches the controller mode. The PLC changes mode bumplessly.
Pump Start / Stop	Operator commands to PMP101.
Reset interlock	Clears a latched low-level trip once the level has recovered.
TREND, Window	Historian trend, 5 minutes by default. Levels on the left axis, OP on the right axis.

The step response is clean. The output kicks up to about 80 %, the level rises to 340 mm in roughly a minute without overshoot, and the output settles at 68.6 %, a little higher than before because a fuller tank 1 drains faster.

4.7 Step 6: open the Faults tab

Click the **Faults** tab. Before anything is injected, **Active faults** reads “Plant is healthy.”

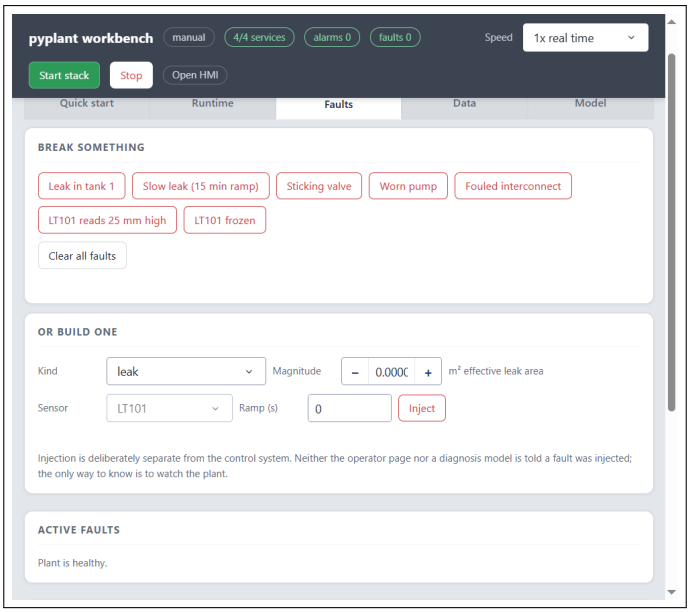


Figure 4.8: Step 6: the Faults tab, nothing injected.

Element	*What it shows or does*
BREAK SOMETHING	One-click presets (red outline). Each injects a fault of a sensible size: **Leak in tank 1**, **Slow leak (15 min ramp)**, **Sticking valve**, **Worn pump**, **Fouled interconnect**, **LT101 reads 25 mm high**, **LT101 frozen**.
Clear all faults	Removes every active fault at once.
OR BUILD ONE: Kind	Fault type, alphabetical: `fouling`, `leak`, `pump_wear`, `sensor_bias`, `sensor_drift`, `sensor_noise`, `sensor_stuck`, `stiction`.
Magnitude (−/+)	Size of the fault. The text on the right gives its unit for the chosen kind (here m^2 of effective leak area).
Sensor	Which transmitter a sensor fault acts on. It is greyed out for process faults.
Ramp (s)	0 for a step; otherwise the time the fault takes to grow to full size.
Inject	Applies the custom fault.
Grey note	Injection is separate from control: neither the HMI nor a model is told about the fault.

4.8 Step 7: inject the leak

Press **Leak in tank 1**. The message under the presets confirms the injection and the header badge turns red at **faults 1**.

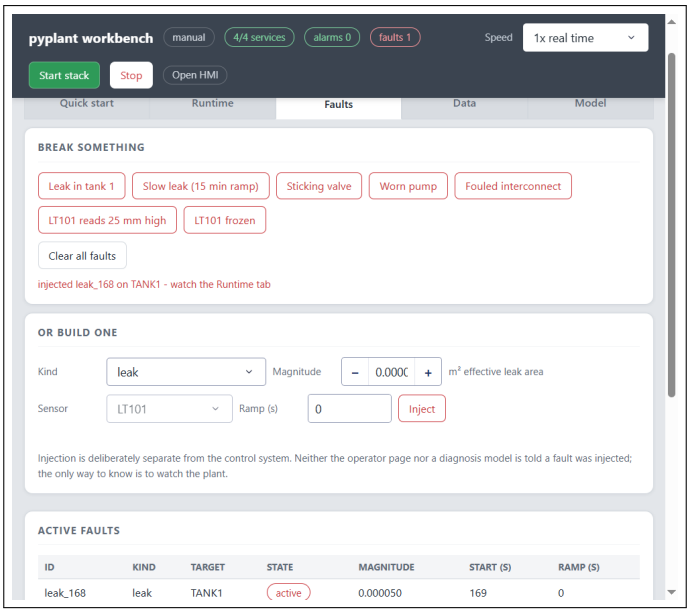


Figure 4.9: Step 7: the leak is injected.

Element	*What it shows or does*
faults 1 (red badge)	One fault is active in the plant.
Message	"injected leak_168 on TANK1 – watch the Runtime tab". The number is the plant time of injection, used as the fault id.
ACTIVE FAULTS row	**ID** leak_168, **KIND** leak, **TARGET** TANK1, **STATE** active, **MAGNITUDE** 0.000050, **START (S)** 169, **RAMP (S)** 0.

4.9 Step 8: check the ground truth

Scroll down. The ground-truth table compares the plant's true state with what each transmitter reports.

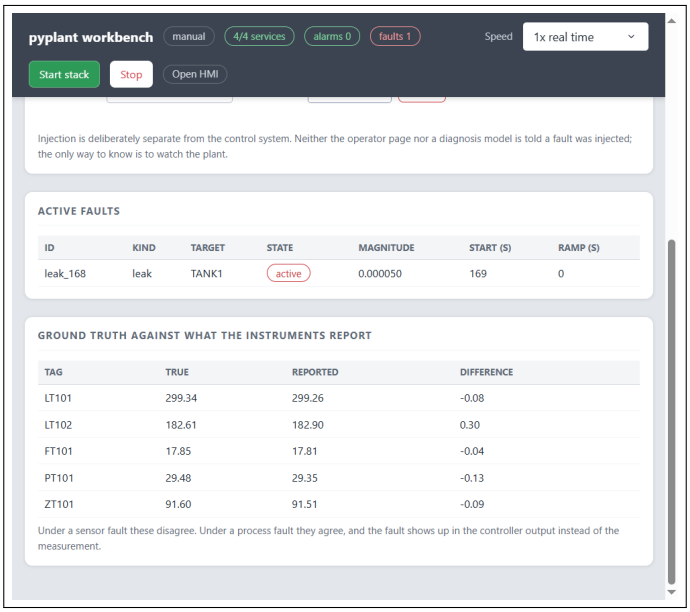


Figure 4.10: Step 8: active fault and ground truth, about 30 s after injection.

Element	*What it shows or does*
TAG	The five measured channels.
TRUE	The physical value inside the simulator.
REPORTED	What the transmitter sends to the PLC, including noise and any sensor fault.
DIFFERENCE	Reported minus true. Here every difference is within the noise (below 0.3), which tells you the fault is in the *process*, not in a sensor. Under **LT101 reads 25 mm high** the LT101 row would show about +25.

Notice the true level, 299.3 mm: the leak has already pulled tank 1 about 40 mm below the 340 mm setpoint.

4.10 Step 9: watch the controller compensate

Go back to **Runtime** and scroll to **LIC101**, about two minutes after the injection.

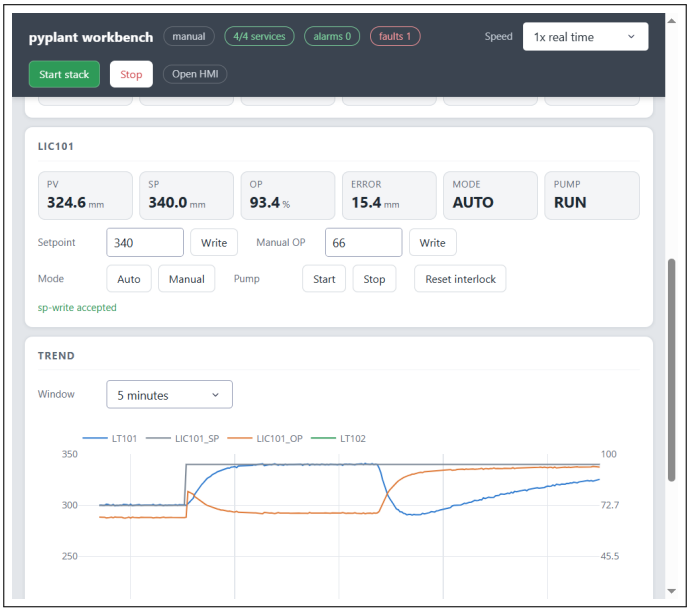


Figure 4.11: Step 9: the loop fighting the leak. The trend shows the setpoint step from step 5, then the leak.

Element	*What it shows or does*
PV 324.6 mm, ERROR 15.4 mm	The level is still below setpoint but climbing back.
OP 93.4 %	The valve is almost fully open. Before the leak it was 68.6 %.
MODE AUTO, PUMP RUN	No interlock trip; the operator sees no alarm.
Trend, first edge	The setpoint step from step 5 (grey) and the clean level response (blue).
Trend, second edge	The leak. The level (blue) drops to about 290 mm while the output (orange) jumps to about 92 % and stays there. The level recovers slowly, because the valve has little travel left.

This is the signature to remember: a *process* fault moves the controller output, while the level only dips and then returns.

4.11 Step 10: see it as the operator, on the HMI

Click **Open HMI** in the header, or open `http://127.0.0.1:5405`. The HMI is a separate service and reads everything over OPC UA.

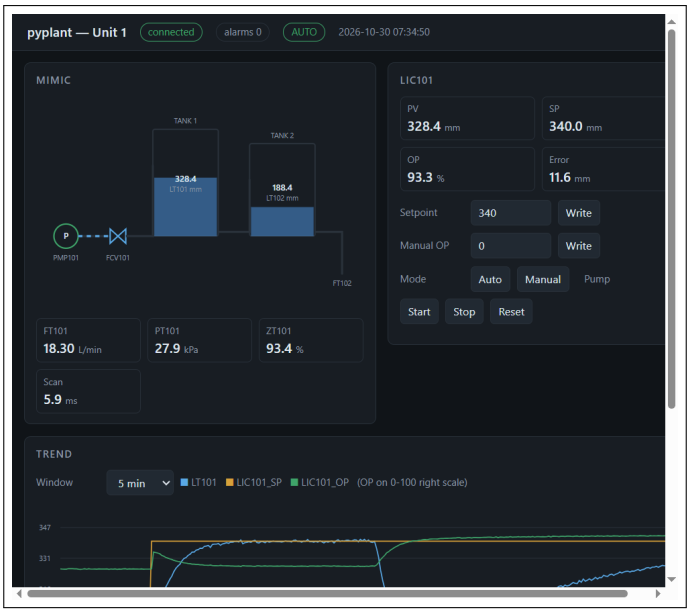


Figure 4.12: Step 10: the operator HMI during the leak.

Element	*What it shows or does*
pyplant – Unit 1	HMI header. **connected** (green) means the OPC UA session is live. It also shows the alarm count, the controller mode badge **AUTO**, and the PLC timestamp.
MIMIC	Same process picture, in a dark ISA-101 style: tank 1 328.4 mm, tank 2 188.4 mm.
FT101 / PT101 / ZT101	Inflow up to 18.30 L/min (from about 10), pump pressure down to 27.9 kPa (the pump runs further out on its curve), valve at 93.4 %.
Scan	PLC scan time, 5.9 ms.
LIC101 panel	Same faceplate as the workbench: PV, SP, OP, Error, plus **Setpoint**/**Manual OP** with **Write**, **Auto**/**Manual** and pump **Start**/**Stop**/**Reset**.
TREND, Window	2 min to 1 hour. The legend notes that OP uses the 0–100 right-hand scale.
ALARMS, Acknowledge all	Further down the page (not in the screenshot). It stays empty in this scenario: no limit is crossed.

Why no alarm?

The level never crossed the low alarm, so a conventional alarm system says nothing. A 5×10^{-5} m^2 leak loses about 8 L/min, nearly as much as the normal throughput, and the only evidence is a valve that is 25 points more open. Catching this is the job of the detectors trained in steps 14–16.

4.12 Step 11: clear the fault and open the Data tab

Return to the workbench. On **Faults** press **Clear all faults**; the header goes back to **faults 0**. Then click the **Data** tab.

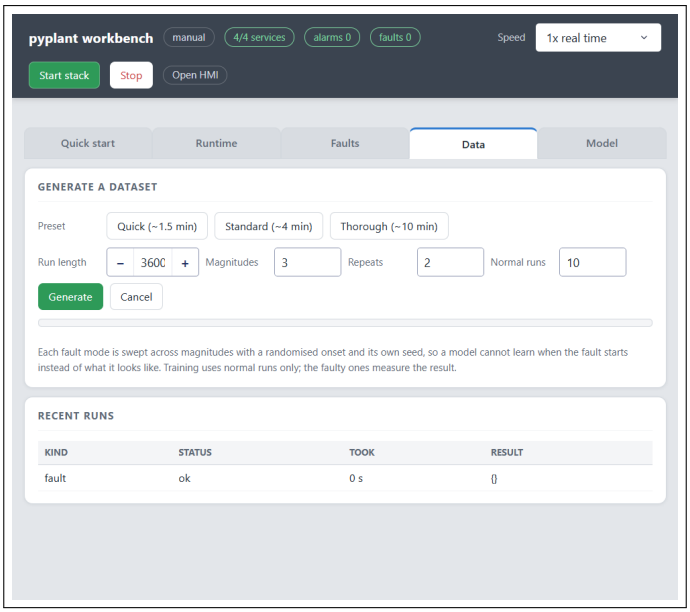


Figure 4.13: Step 11: the Data tab with its default settings.

Element	*What it shows or does*
Preset: Quick / Standard / Thorough	Fill the four fields below with a ready-made size: about 1.5, 4 or 10 minutes of computing.
Run length (−/+)	Plant seconds per simulated run (default 3600).
Magnitudes	How many fault sizes per fault mode.
Repeats	How many random onsets per size.
Normal runs	Fault-free runs. Training uses only these.
Generate / Cancel	Start or stop the background job.
Progress bar	Grey bar under the buttons. It fills blue as runs finish.
RECENT RUNS	Job history: **KIND**, **STATUS**, **TOOK** and a **RESULT** summary. The “fault” row is the **Clear all faults** action.

4.13 Step 12: generate a Quick dataset

Press **Quick (~1.5 min)**. The fields change to run length 1200, magnitudes 1, repeats 1, normal runs 6. Press **Generate**.

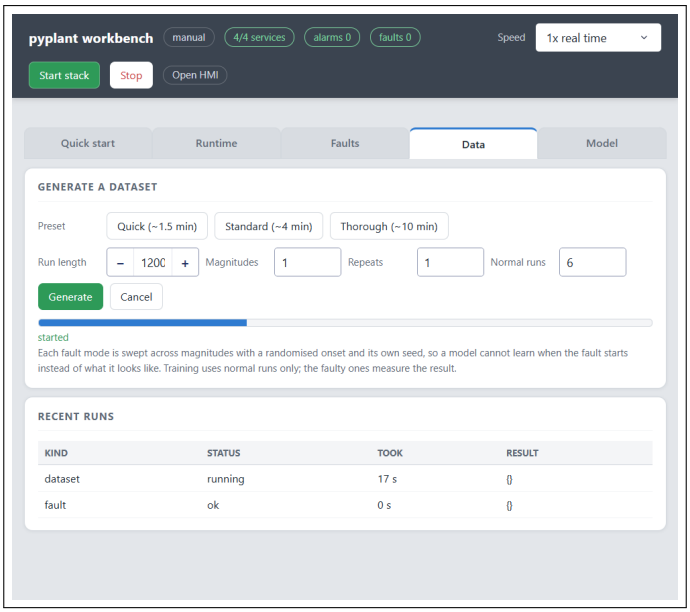


Figure 4.14: Step 12: dataset generation in progress.

Element	*What it shows or does*
started	Green confirmation under the bar.
Progress bar	About a third full: runs completed out of runs planned.
dataset / running / 17 s	The new job, still running, with elapsed time.

The live stack keeps running meanwhile. Dataset runs use the offline fast runner in a separate process and do not touch the live plant.

4.14 Step 13: the dataset is ready

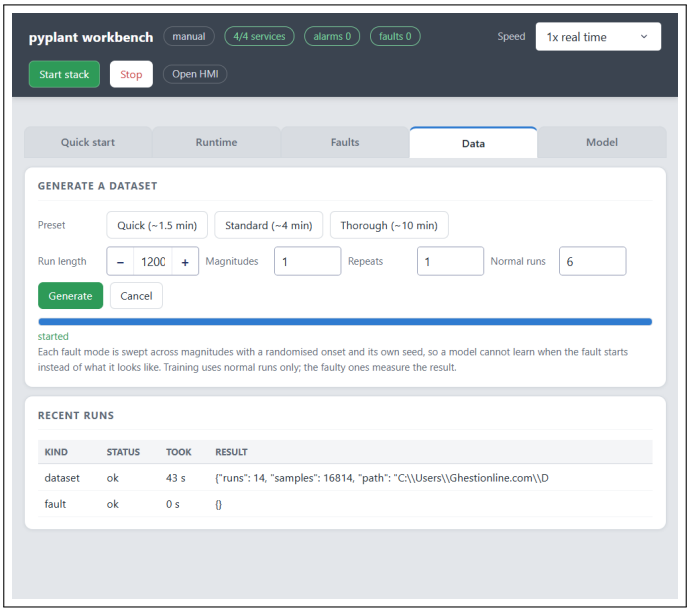


Figure 4.15: Step 13: dataset finished.

Element	*What it shows or does*
Full bar	The job has finished.
dataset / ok / 43 s	Status and wall time.
RESULT	`{"runs": 14, "samples": 16814, "path": ...}`: 6 normal plus 8 faulty runs, written to `work/manual/dataset.parquet` (1.7 MB).

4.15 Step 14: set up training

Click the **Model** tab.

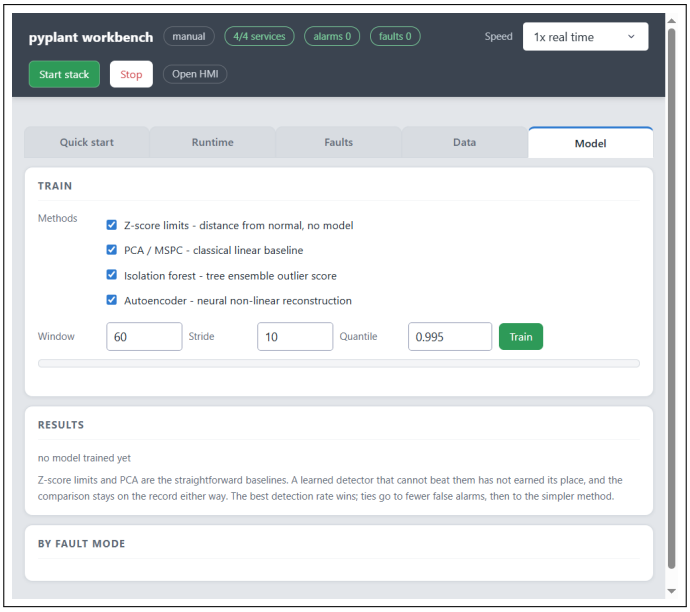


Figure 4.16: Step 14: the Model tab before training.

Element	*What it shows or does*
Methods (checklist)	The four detectors, from simplest to most complex: Z-score limits, PCA/MSPC, Isolation forest, Autoencoder. Untick any you do not want to train.
Window	Samples per feature window (60).
Stride	Step between windows (10).
Quantile	The alarm threshold is this quantile of the scores on normal data (0.995). Higher means fewer false alarms but later detection.
Train	Starts the training job.
RESULTS / BY FAULT MODE	Empty until the first model is trained (“no model trained yet”).

4.16 Step 15: train and read the results

Keep the defaults and press **Train**. The message reads “started: zscore, pca, isolation_forest, autoencoder”. On the Quick dataset the job takes about 12 s, and the bar fills when it is done.

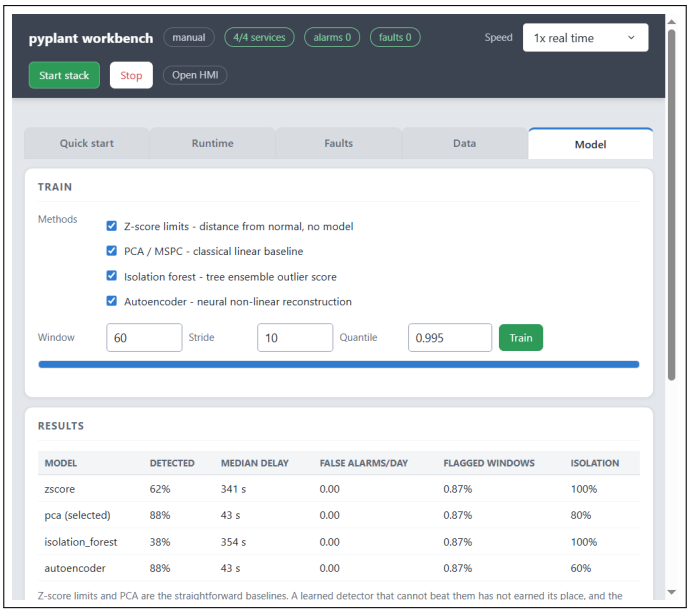


Figure 4.17: Step 15: four detectors compared.

Element	*What it shows or does*
model	Method name. The deployed one is marked **(selected)**.
detected	Share of faulty test runs flagged after onset: z-score 62 %, PCA 88 %, isolation forest 38 %, autoencoder 88 %.
median delay	Time from onset to first alarm (43 s for PCA and the autoencoder; more than 340 s for the others).
false alarms/day	Alarms on normal test runs, scaled to a day: 0.00 for all four.
flagged windows	Share of normal windows over the threshold (0.87 %).
isolation	How often the top-scoring channel was the expected one.

PCA and the autoencoder tie on detection and false alarms, so the rule in the grey note picks the *simpler* method: PCA is deployed to `artifacts/detector.joblib`.

4.17 Step 16: check the leak, fault mode by fault mode

Scroll to **BY FAULT MODE**. It shows the selected detector's results for each fault mode.

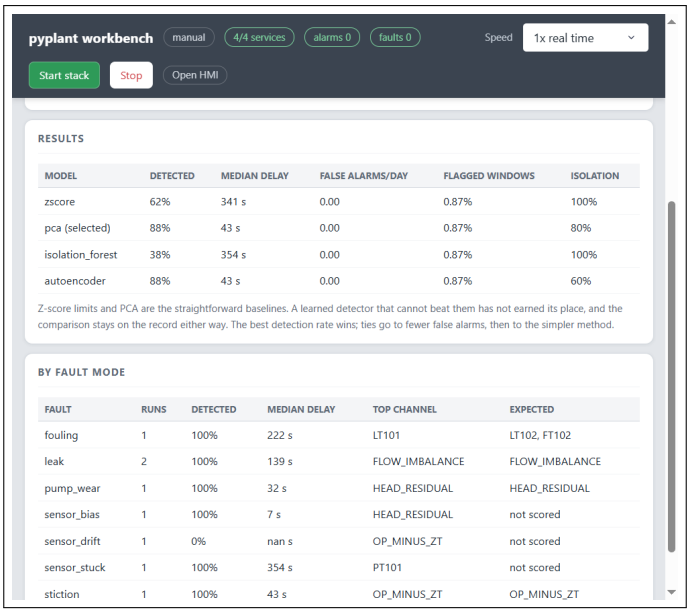


Figure 4.18: Step 16: per-fault-mode breakdown of the selected detector.

Element	*What it shows or does*
fault / runs	Fault mode and how many test runs it had.
detected / median delay	The **leak** row: 2 runs, 100 % detected, median 139 s after onset.
top channel	Channel with the largest contribution at the first alarm. For the leak it is FLOW_IMBALANCE, the inflow-minus-outflow residual.
expected	Channel physics says should respond. The leak's matches, so the detector also *isolates* it correctly. "not scored" rows are sensor faults, where isolation is not meaningful.

4.18 Step 17: stop the stack

Press **Stop** in the header. The badge returns to **0/4 services**, the message reads "stack stopped", and every service shows **down**.

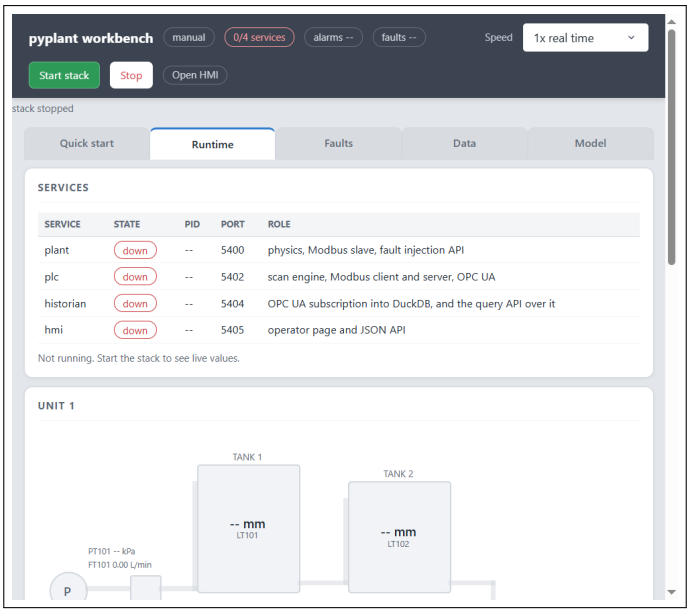


Figure 4.19: Step 17 (final): the stack stopped.

Element	*What it shows or does*
stack stopped	Confirmation.
0/4 services,	Nothing is running any more.
alarms –, faults –	
Mimic "– mm"	No live values without the PLC.

Everything from the session stays in `work/manual`: `config.json` (the settings used), `history.duckdb` (the historian), `dataset.parquet`, `artifacts/` (all four detectors, the deployed `detector.joblib`, `metrics.json` and per-run CSVs), and `jobs/` (job records and service logs).

4.19 Numbers to expect

Step	Quantity	Observed
1	OP change for the default leak (offline)	+9.6 %
4	Healthy steady state at SP 300	LT101 300.0, OP 66 %, FT101 10.1
5	After SP step to 340	PV 339.7, OP 68.6 %, no overshoot
9	Two minutes into the leak	PV 324.6, OP 93.4 %, no alarm
10	HMI during the leak	FT101 18.3 L/min, PT101 27.9 kPa
13	Quick dataset	14 runs, 16 814 samples, 43 s
15	Selected detector	PCA, 88 % detected, 0 false alarms/day
16	Leak, per mode	100 % detected, 139 s, FLOW_IMBALANCE

Table 4.1: Reference values from this walkthrough. Expect small run-to-run differences from noise and random fault onsets.

Chapter 5

Using the toolbox step by step

This chapter walks through the toolbox in the order the workflow cards suggest. Figure 5.1 is the whole procedure as a flow diagram; each tutorial below covers one part of it.

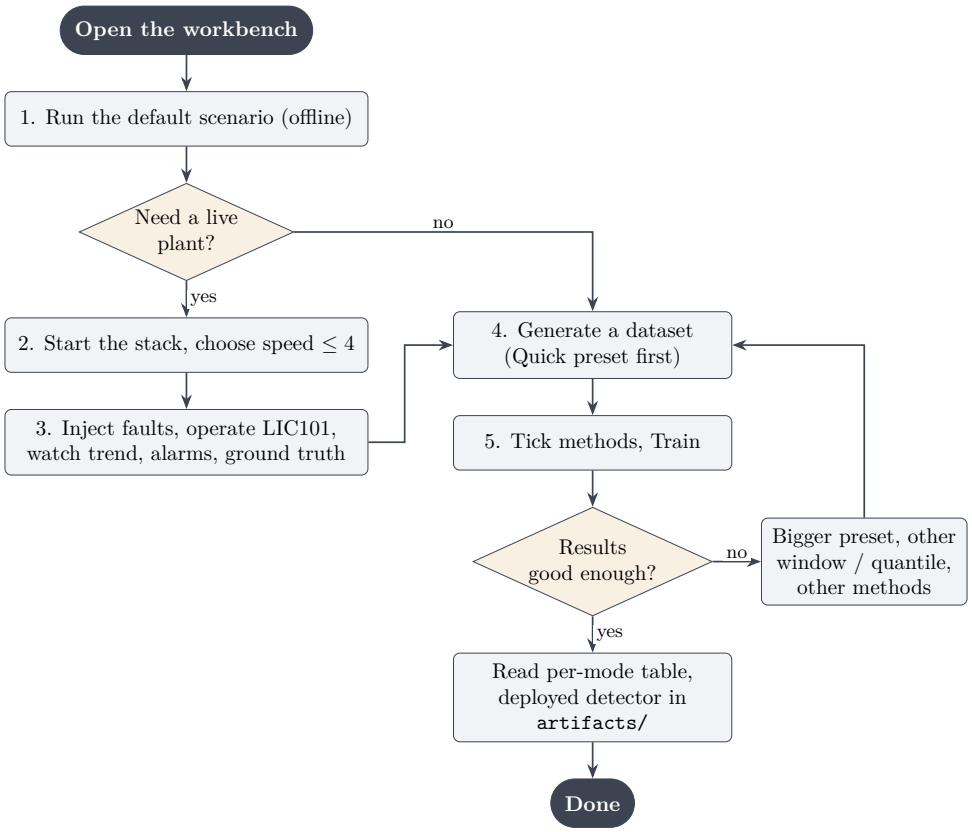


Figure 5.1: Recommended workflow through the toolbox.

5.1 Tutorial 1: the default scenario

1. Start the workbench and open `http://127.0.0.1:8050`. The Quick start tab opens and the default leak scenario runs by itself.
2. Read the summary cards. With the setpoint held, LT101 ends where it started (±0.1 mm) while the controller output rises by about 9.6%. This is the central fact of the project: in a closed loop the controller absorbs the fault, so the measurement alone does not show it.
3. Choose **Sticking control valve**, length **1 hour**, and press **Run scenario**. After onset

the level starts to hunt and the orange controller output and the green stem position stop agreeing: the valve is stuck until the command moves far enough, then jumps.

4. Choose **LT101 frozen**, tick **setpoint moves**, and run. After the setpoint changes the controller chases a reading that cannot follow, winds the valve shut, starves the pump, and the minimum-flow interlock trips. The **Pump trip** card shows when.

5.2 Tutorial 2: the live plant

1. Set **Speed** to 1× or 2× (at most 4× on Windows) and press **Start stack**. After about ten seconds the services pill reads **4/4 services**.
2. Open the Runtime tab. The plant starts lined out at its design point: LT101 ≈ 300 mm, valve ≈ 66%, flow ≈ 10 L/min, pump pressure ≈ 50 kPa.
3. **Setpoint step.** Type 350 in **Setpoint** and press **Write**. The level rises to 350 mm in about 40 s with no overshoot (IMC tuning, Section 7.3).
4. **Manual mode.** Press **Manual**, write 50 to **Manual OP**. The valve closes to 50% and the level falls; the controller is not acting. Press **Auto**: the output continues from 50% without a jump (bumpless transfer) and the level returns to setpoint.
5. **Rejected write.** Write 3000 to **Setpoint**. The HMI refuses it with an error message instead of clamping it silently.
6. **Interlock.** Press pump **Stop** and then **Start** while watching the alarms, or raise the setpoint to 560 mm so the level crosses the 550 mm trip. The interlock latches, the pump stops and the valve closes. Restore a normal setpoint and press **Reset interlock**.

5.3 Tutorial 3: faults on the live plant

1. Open the Faults tab and press **Leak in tank 1**. The active-fault list shows the new fault; the ground-truth table shows readings equal to the true values, because a leak is a process fault and the instruments are honest.
2. On the Runtime tab, the controller output climbs by about 10% over a minute while the level holds.
3. Press **Clear all faults**, then **LT101 reads 25 mm high**. Now the ground-truth table shows LT101 reported 25 mm above its true value. The controller drives the *reading* back to 300 mm, so the true level sits 25 mm low and LT102 falls.
4. Try **Sticking valve** at 4× speed for a few minutes and watch LIC101_OP and ZT101 on the mimic diverge.
5. Use **Or build one** to inject, for example, `pump_wear` with magnitude 0.3 and ramp 300 s, and watch PT101 fall below what the pump curve predicts.

5.4 Tutorial 4: a dataset

1. Open the Data tab and press **Quick (~1.5 min)**. The fields become 1200 s, 1 magnitude, 1 repeat, 6 normal runs.
2. Press **Generate**. The progress bar fills; when it finishes, **Recent runs** shows a `dataset` row with the number of runs and samples, and `dataset.parquet` appears in the workspace.
3. For publishable numbers use **Standard** (58 runs of an hour) or **Thorough**.

The dataset is generated offline, in plant time, with the same code as the live plant (Figure 1.3). Each row is one second of one run and carries all signals, the label, the fault kind, target, onset and ramp.

5.5 Tutorial 5: training and comparing detectors

1. Open the Model tab. Leave all four methods ticked, window 60, stride 10, quantile 0.995, and press **Train**.
2. When the bar is full, the Results table compares the methods. The method with the highest detection rate is deployed; ties go to the one with fewer false alarms and then to the simpler method.
3. Read the **By fault mode** table. For leak, pump wear, stiction and fouling the **expected** column names the physics channel the fault should implicate; a match is an isolated fault.
4. The deployed detector is `artifacts/detector.joblib` with its `manifest.json`; every method's detector and per-run CSV are kept beside it.

On the Quick dataset, a typical result is:

Method	Detected	Median delay	False alarms/day	Isolation
Z-score limits	62.5%	341 s	0.00	100%
PCA / MSPC	87.5%	43 s	0.00	80%
Isolation forest	37.5%	354 s	0.00	100%
Autoencoder	87.5%	43 s	0.00	60%

On the Standard dataset the README reports the autoencoder at 100% detection with a 70 s median delay and PCA at 83% with 256 s. Small datasets favour simple models; larger ones let the learned model pull ahead.

5.6 Tutorial 6: the same pipeline from Python

Everything the workbench does is available as functions:

```
from pyplant.workbench.tasks import (run_default_scenario,
                                     generate_dataset, train_detector)

result = run_default_scenario("stiction", duration=3600.0)
print(result["onset"], result["series"]["LIC101_OP"][-1])

noop = lambda fraction, message="": None # progress callback
generate_dataset(noop, "./work/scratch", duration=1200.0,
                 magnitudes=1, repeats=1, normal_runs=6)
summary = train_detector(noop, "./work/scratch",
                         methods=["zscore", "pca"])
print(summary["selected"], summary["results"]["pca"]["detection_rate"])
```

And the command-line training pipeline of the README:

```
python -m pyplant.ml.train --out artifacts --duration 3600 \
      --magnitudes 3 --repeats 2 --normal-runs 6 --window 60 --stride 10
```

Chapter 6

The plant model

`pyplant/core/hybrid_tank.py` holds the physics as pure functions: SI units throughout, no sockets, no sleeping, no global state. This chapter derives every equation it implements.

6.1 State, inputs and outputs

The state vector is

$$\boldsymbol{x} = \begin{bmatrix} h_1 & h_2 & z \end{bmatrix}^\top,$$

the two levels in metres and the valve stem position $z \in [0, 1]$. The inputs are the valve command $u \in [0, 1]$ and the pump run command. The outputs are the instrument readings of Table 6.1.

Tag	Quantity	Formula	Span
LT101	tank 1 level	$1000\,h_1$	0–600 mm
LT102	tank 2 level	$1000\,h_2$	0–600 mm
FT101	pump discharge flow	$60\,000\,q_{\text{in}}$	0–20 L/min
FT102	tank 2 outlet flow	$60\,000\,q_{\text{out}}$	0–20 L/min
PT101	pump discharge pressure	$p_{\text{pump}}/1000$	0–100 kPa
ZT101	valve stem position	$100\,z$	0–100 %
PMP101_RUN	pump running feedback	pump command	bool

Table 6.1: Measurements. `measure()` is the only function that converts SI to engineering units.

6.2 Mass balances

Each tank is a vertical cylinder of cross-section A. Conservation of volume for an incompressible liquid gives

$$A_1 \frac{dh_1}{dt} = q_{\text{in}} - q_{12} - q_{\text{leak}}, \tag{6.1}$$

$$A_2 \frac{dh_2}{dt} = q_{12} - q_{\text{out}}. \tag{6.2}$$

Both tanks are bounded: a level at the overflow lip $h_{\max} = 0.6\,\text{m}$ cannot rise further, and an empty tank cannot drain.

6.3 Gravity drains: Torricelli's law

Flow through an orifice under a head h follows from Bernoulli's equation between the free surface and the jet, $v = \sqrt{2gh}$, multiplied by an effective discharge area C_dA that lumps the geometric area and the contraction coefficient:

$$q = C_d A \sqrt{2gh}. \tag{6.3}$$

This gives the interconnect, outlet and leak flows:

$$q_{12} = C_d A_{12}\, g_{12} \sqrt{2gh_1}, \qquad q_{\text{out}} = C_d A_{2,\text{out}} \sqrt{2gh_2}, \qquad q_{\text{leak}} = C_d A_{\text{leak}} \sqrt{2gh_1}, \tag{6.4}$$

where $g_{12} \le 1$ is the fouling factor and $C_dA_{\text{leak}} \ge 0$ the leak area. Both are healthy at $g_{12} = 1$, $C_dA_{\text{leak}} = 0$.

6.4 Pump and valve: an algebraic loop solved in closed form

A centrifugal pump develops less pressure as it delivers more flow. A quadratic curve is the standard approximation:

$$p_{\text{pump}} = \Delta p_0\, \gamma \left(1 - \left(\frac{q}{q_{\max}}\right)^2\right), \tag{6.5}$$

with shut-off pressure $\Delta p_0 = 60\,\text{kPa}$, run-out flow $q_{\max} = 25\,\text{L/min}$ and wear factor $\gamma \le 1$.

The control valve passes a flow proportional to the square root of the pressure drop across it. The drop is the pump pressure minus the static head of tank 1:

$$q = k\sqrt{p_{\text{pump}} - \rho g h_1}, \qquad k = C_v\, \phi(z). \tag{6.6}$$

The valve has an *equal-percentage* inherent characteristic with rangeability $R = 30$: equal increments of stem travel change the flow by equal percentages,

$$\phi(z) = R^{\,z-1}, \qquad \phi(1) = 1,\ \phi(0) = 1/R, \tag{6.7}$$

and below a seat position of 0.5% the valve shuts tight. Under a falling pump curve, an equal-percentage valve gives a roughly linear *installed* characteristic, which is why it is chosen.

Pump pressure depends on flow, and flow depends on pump pressure. Substituting one into the other and solving the quadratic gives

$$\boxed{q_{\text{in}} = k\sqrt{\frac{\Delta p_0 \gamma - \rho g h_1}{1 + k^2\, \Delta p_0 \gamma / q_{\max}^2}}} \tag{6.8}$$

so no iteration is needed and the system is not stiff at a 50 ms step. If the valve is shut the pump dead-heads at $\Delta p_0\gamma$; if the pump is off, $q_{\text{in}} = 0$.

6.5 Valve actuator

The stem follows the command through a first-order lag with $\tau_v = 2\,\text{s}$:

$$\tau_v \frac{dz}{dt} = u - z. \tag{6.9}$$



(a) Inherent valve characteristic, $R = 30$.

(b) Pump curve, healthy and worn.

Figure 6.1: Component characteristics used by the plant model.

6.6 Numerical integration

The model is integrated with the classical fourth-order Runge–Kutta method at $\Delta t = 50\,\text{ms}$:

$$\begin{aligned} \boldsymbol{k}_1 &= f(\boldsymbol{x}_n, u), & \boldsymbol{k}_2 &= f(\boldsymbol{x}_n + \tfrac{\Delta t}{2}\boldsymbol{k}_1, u), \\ \boldsymbol{k}_3 &= f(\boldsymbol{x}_n + \tfrac{\Delta t}{2}\boldsymbol{k}_2, u), & \boldsymbol{k}_4 &= f(\boldsymbol{x}_n + \Delta t\,\boldsymbol{k}_3, u), \\ \boldsymbol{x}_{n+1} &= \boldsymbol{x}_n + \tfrac{\Delta t}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4), \end{aligned}$$

after which levels are clipped to $[0, h_{\max}]$ and z to $[0, 1]$. The input is held constant within a step.

6.7 Steady state

At equilibrium $dh_1/dt = 0$. For a fixed valve position, inflow falls and outflow rises monotonically with h_1, so the residual $r(h_1) = q_{\text{in}} - q_{12} - q_{\text{leak}}$ has a single root, found by bisection. Then $dh_2/dt = 0$ gives tank 2 directly:

$$h_2 = \frac{1}{2g}\left(\frac{q_{12}}{C_d A_{2,\text{out}}}\right)^2. \tag{6.10}$$

`steady_state()` is used to start the plant lined out and by the tests.

6.8 Sizing and the design point

At the design point the valve is 66% open, the level is 300 mm, the flow 10 L/min and the pump pressure 50 kPa. The usable level range is roughly 30 to 580 mm, corresponding to valve positions from 0.30 to 0.79.

6.9 Linearisation and the process time constant

Tuning needs a linear model. Around an operating point $(\bar{h}_1, \bar{z})$, write $h_1 = \bar{h}_1 + \delta h$, $z = \bar{z} + \delta z$ and expand equation (6.1) to first order:

$$A_1 \frac{d\,\delta h}{dt} = \underbrace{\left(\frac{\partial q_{\text{in}}}{\partial h_1} - \frac{\partial q_{12}}{\partial h_1}\right)}_{-1/R_h} \delta h + \frac{\partial q_{\text{in}}}{\partial z}\,\delta z. \tag{6.11}$$

Inflow falls as the back-pressure rises, so $\partial q_{\text{in}}/\partial h_1 < 0$, and outflow rises with head, $\partial q_{12}/\partial h_1 = q_{12}/(2h_1) > 0$; the bracket is therefore negative and the tank is self-regulating. This is a first-order system

$$\tau \frac{d\,\delta h}{dt} + \delta h = K\,\delta z, \qquad \tau = A_1 R_h, \quad K = R_h \frac{\partial q_{\text{in}}}{\partial z}. \tag{6.12}$$

Parameter (`Params` field)	Value	Meaning
`area_tank1`, `area_tank2`	$0.020\,\mathrm{m}^2$	tank cross-sections (≈160 mm diameter)
`height_max`	$0.600\,\mathrm{m}$	overflow lip
`pump_dp0`	$60\,000\,\mathrm{Pa}$	shut-off pressure
`pump_qmax`	$4.17 \times 10^{-4}\,\mathrm{m}^3/\mathrm{s}$	run-out flow, 25 L/min
`valve_cv`	2.429×10^{-6}	valve coefficient fully open, $\mathrm{m}^3/(\mathrm{s}\sqrt{\mathrm{Pa}})$
`valve_rangeability`	30	equal-percentage R
`valve_seat`	0.005	tight shut-off below this position
`tau_valve`	$2.0\,\mathrm{s}$	actuator lag
`cda_12`	$6.90 \times 10^{-5}\,\mathrm{m}^2$	tank 1 to tank 2 orifice
`cda_2out`	$9.00 \times 10^{-5}\,\mathrm{m}^2$	tank 2 drain
`pump_gain`	1.0	fault knob: < 1 is impeller wear
`cda_12_gain`	1.0	fault knob: < 1 is fouling
`cda_leak`	0.0	fault knob: leak area

Table 6.2: Plant parameters and their defaults.

An open-loop step test on the model at the design point gives $K \approx 15.4\,\mathrm{mm}$ per % of valve travel and $\tau \approx 72\,\mathrm{s}$, the values the controller is tuned for. The valve lag (2 s) is small compared with τ and is neglected in tuning.

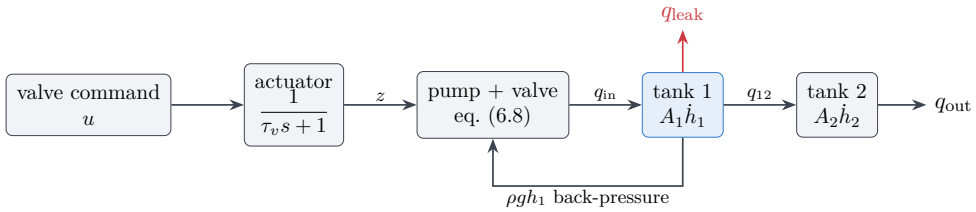


Figure 6.2: Signal flow inside the plant model.

6.10 The simulation stepper

`pyplant/runners/base.py` holds `Simulation`, the one loop body both runners drive. It has three deliberately different rates:

Rate	Default	What it governs
`dt`	50 ms	integration accuracy
`control_period`	100 ms	how often the controller sees the plant
`log_period`	1 s	what the historian and datasets store

The controller period must be a whole number of integration steps, and time is computed from an integer step count rather than by repeated addition, so it never accumulates rounding error. Between controller executions the inputs are held: a genuine zero-order hold, as on a real PLC. Faults are applied in `Simulation.integrate()`, the single place both runners share (Chapter 10).

Chapter 7

Control theory and the LIC101 loop

This chapter explains feedback control from first principles and then the specific controller in `pyplant/plc/pid.py`: a velocity-form PID with structural anti-windup, bumpless transfer, derivative on filtered measurement and IMC tuning.

7.1 Feedback control

A feedback loop measures the controlled variable (the *process value*, PV), compares it with the desired value (the *setpoint*, SP), and moves a manipulated variable (the *controller output*, OP) to remove the difference $e = \mathrm{SP} - \mathrm{PV}$. Figure 7.1 shows LIC101 in this form.

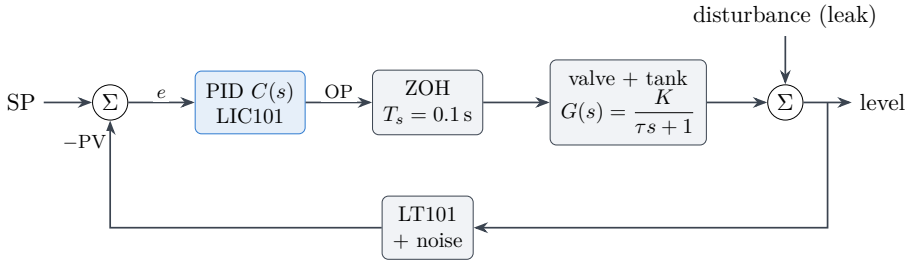


Figure 7.1: LIC101 as a feedback loop. The PLC samples the level every 100 ms and holds its output between scans.

7.2 The PID law

The ideal continuous PID controller in standard (ISA) form is

$$u(t) = K_c \left(e(t) + \frac{1}{T_i} \int_0^t e(\tau)\, d\tau + T_d \frac{de}{dt} \right), \tag{7.1}$$

with controller gain K_c, integral time T_i and derivative time T_d. Each term has a role:

Proportional reacts to the present error. Alone it leaves a steady offset whenever a non-zero output is needed to hold the setpoint.

Integral accumulates past error and keeps moving the output until the error is exactly zero. It removes offset, and it is the reason a leak is masked (Section 7.5).

Derivative anticipates by reacting to the rate of change. On a noisy level it does more harm than good, so LIC101 runs with $T_d = 0$.

7.2.1 Positional and velocity forms

Discretising at sample time T_s with backward differences gives the *positional* form

$$u_k = K_c\Big(e_k + \frac{T_s}{T_i}\sum_{j\le k} e_j + \frac{T_d}{T_s}(e_k - e_{k-1})\Big). \tag{7.2}$$

Subtracting u_{k-1} from u_k gives the *velocity* (incremental) form used here:

$$\Delta u_k = K_c(e_k - e_{k-1}) + K_c\frac{T_s}{T_i}e_k - K_c\frac{T_d}{T_s}(y^f_k - 2y^f_{k-1} + y^f_{k-2}), \qquad u_k = \mathrm{clip}\,(u_{k-1} + \Delta u_k,\, 0,\, 100), \tag{7.3}$$

where y^f is the filtered measurement (below). For a reverse-acting loop the derivative term has a minus sign because it acts on the measurement.

7.2.2 Why the velocity form

In the velocity form the output itself is the integrator state. Two properties that need extra logic in the positional form become automatic:

Anti-windup is structural

When the actuator saturates, a positional PID keeps integrating the error, so the integral grows far beyond the limit (*windup*) and the output stays pinned long after the error reverses. In equation (7.3) the increment that would push past a limit is simply clipped away. No hidden accumulator grows, and the output leaves the limit on the first scan the error changes sign.

Bumpless transfer is structural

In manual mode the output is whatever the operator writes, and the controller keeps its error history fresh against the current PV (`track()`). On return to auto it continues incrementing from the manual value, so the valve does not jump.

7.2.3 Derivative on filtered measurement

Taking the derivative of the error differentiates the setpoint too, so a setpoint step would produce an impulse (*derivative kick*). The code differentiates the measurement instead. The measurement is first passed through a first-order low-pass filter with time constant $T_f = 5\,\mathrm{s}$, discretised as

$$y^f_k = y^f_{k-1} + \alpha\,(y_k - y^f_{k-1}), \qquad \alpha = \frac{T_s}{T_f + T_s}. \tag{7.4}$$

7.2.4 Initialisation from valve feedback

A velocity PID increments from wherever its output already is. Starting at 0% would slam shut the valve of a plant running at 66%, dead-head the pump and trip the interlock on the first scan. On the first scan the program therefore seeds the output from the measured stem position ZT101.

7.3 IMC tuning

Internal Model Control (IMC) tuning chooses the controller so that the closed loop behaves like a first-order lag with a time constant λ the engineer specifies. For a first-order process

$$G(s) = \frac{K}{\tau s + 1}, \tag{7.5}$$

the IMC controller is $Q(s) = G^{-1}(s)\,\frac{1}{\lambda s+1}$ and the equivalent feedback controller is

$$C(s) = \frac{Q}{1 - GQ} = \frac{\tau s + 1}{K\lambda s} = \frac{\tau}{K\lambda}\left(1 + \frac{1}{\tau s}\right), \tag{7.6}$$

which is a PI controller with

$$\boxed{K_c = \frac{\tau}{K\lambda}, \qquad T_i = \tau.} \tag{7.7}$$

The integral time cancels the process pole, and the closed-loop response to a setpoint step is

$$\frac{\mathrm{PV}(s)}{\mathrm{SP}(s)} = \frac{GC}{1 + GC} = \frac{1}{\lambda s + 1}. \tag{7.8}$$

For LIC101, $K = 15.4\,\mathrm{mm/\%}$, $\tau = 72\,\mathrm{s}$ and $\lambda = 13\,\mathrm{s}$:

$$K_c = \frac{72}{15.4 \times 13} \approx 0.36 \;\rightarrow\; 0.35\ \%/\mathrm{mm}, \qquad T_i = 72\ \mathrm{s}, \qquad T_d = 0.$$

The ratio $\tau/\lambda \approx 5.5$ gives a closed loop five times faster than the open loop while leaving margin for the neglected valve lag, the sample time and the nonlinearity.

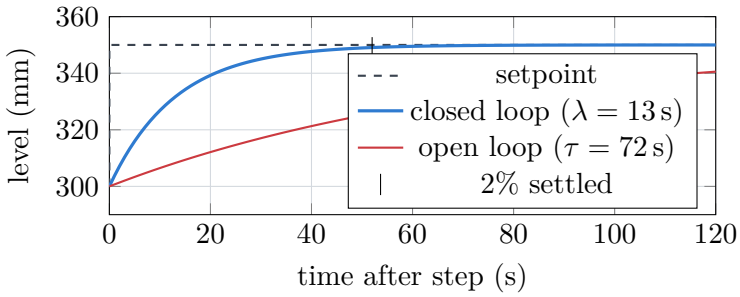


Figure 7.2: Ideal response to a 50 mm setpoint step. The measured response of the real loop matches: no overshoot, 2% settling in about 40–50 s, zero offset.

7.4 Sampling and the zero-order hold

The PLC executes every $T_s = 100\,\mathrm{ms}$ and holds its output until the next scan. A zero-order hold behaves approximately like a dead time of $T_s/2 = 50\,\mathrm{ms}$, negligible against $\lambda = 13\,\mathrm{s}$, but it is modelled physically rather than ignored: the simulator integrates at 50 ms with the input held between scans. A PID that looks better in simulation than on a PLC usually lacks exactly this.

7.5 Why integral action hides a leak

A leak is a load disturbance d on tank 1. With a PI controller the steady-state error to a step disturbance is, by the final value theorem,

$$e_\infty = \lim_{s\to 0} s \cdot \frac{G_d(s)}{1 + G(s)C(s)} \cdot \frac{d}{s} = 0, \tag{7.9}$$

because $C(s)$ has a pole at $s = 0$. The controller drives the level back to setpoint exactly, and the only lasting evidence is that the output has moved:

	LT101	LIC101_OP
before the leak	300.00 mm	66.18%
after the leak	300.00 mm	75.80%

A diagnosis model fed only the process value cannot see this fault. This is why the detectors are given the controller output, the valve position and derived physics channels (Chapter 11).

7.6 Controller parameters

`PidTuning` field	Default	Unit	Meaning
`kc`	0.35	%/mm	controller gain
`ti`	72.0	s	integral time (0 disables integral action)
`td`	0.0	s	derivative time (0 disables derivative)
`tf`	5.0	s	derivative filter time constant
`out_lo`, `out_hi`	0, 100	%	output limits
`direct_acting`	False		False: output rises when PV falls

Chapter 14 explains how to change them.

Chapter 8

The soft PLC

A programmable logic controller is a computer built to run one program forever, deterministically, on a fixed cycle. This chapter describes the PLC model pyplant implements: the scan cycle, the program and its precedence rules, protective logic, timers, the drift-free scheduler and the communication layers.

8.1 The scan cycle

Every PLC scan does three things, always in this order:

1. **Read inputs.** All field inputs are copied into an *input image table* in one operation.
2. **Execute logic.** The program runs against the frozen image. It never reads a field value directly, so every rung in one scan sees the same consistent snapshot.
3. **Write outputs.** The *output image table* is written to the field in one operation.

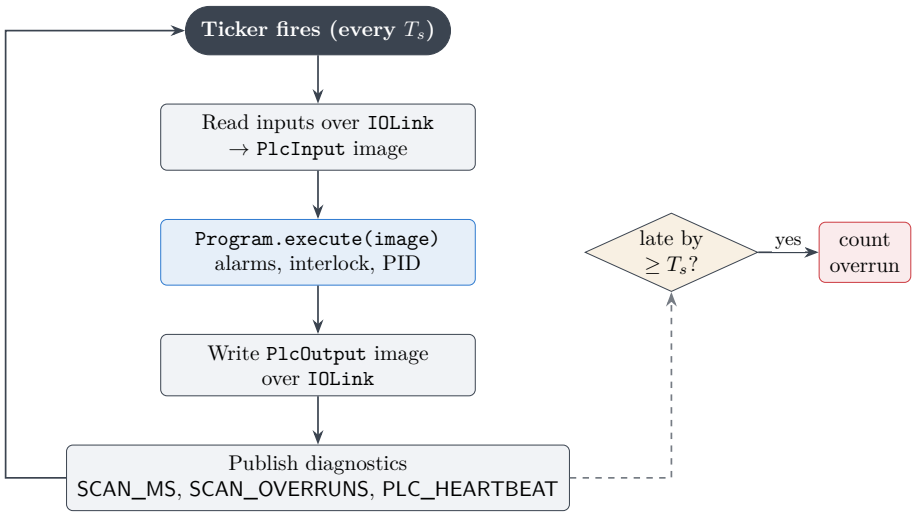


Figure 8.1: One scan of the PLC engine (`pyplant/plc/scan.py`).

The engine (`ScanEngine`) does not know whether its I/O is in-process or Modbus: all I/O goes through the `IOLink` protocol. The program (`Program`) does not know about I/O at all: `execute()` is a pure function of the input image and the program's retained state, which is what makes it testable scan by scan.

8.2 The program

8.2.1 Precedence

Highest first, as on a real unit:

1. **Interlock.** Latched on a trip condition. Forces the pump off, drives the valve shut, and keeps the PID tracking 0% so a reset cannot slam the valve open. Clears only when the operator resets *and* the condition has gone.
2. **Manual.** The operator's output demand passes through; the PID tracks it so that returning to auto is bumpless.
3. **Auto.** The PID holds LT101 at LIC101_SP.

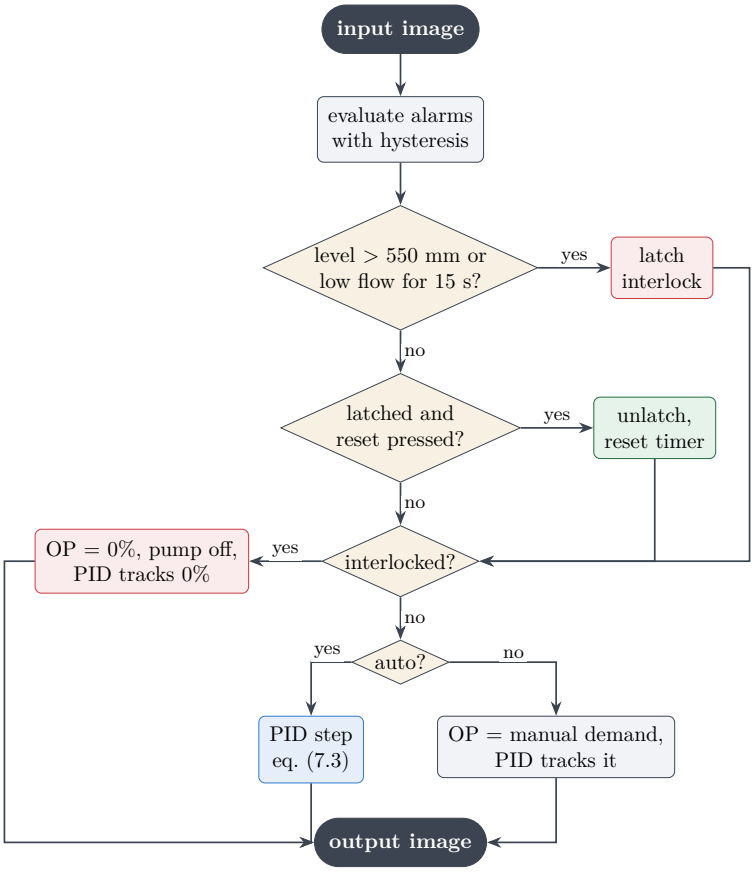


Figure 8.2: Decision flow of `Program.execute()` in one scan.

8.2.2 Protective functions

Function	Trigger	Action
ALM_LT101_HI	LT101 > 500 mm	alarm, priority medium
ALM_LT101_LOLO	LT101 < 50 mm	alarm, priority high
ALM_PT101_HIHI	PT101 > 58 kPa	alarm, priority high
ILK_PMP101	LT101 > 550 mm, or FT101 < 1 L/min for 15 s while running	latched trip: pump off, valve shut; priority critical

All thresholds are read from the tag dictionary, so a respanned instrument cannot leave a stale limit behind in the logic.

Why minimum flow and not high pressure

A centrifugal pump against a shut valve rises to its shut-off head, 60 kPa here. A high-pressure trip below that would fire during any legitimate low-output excursion. The real hazard of dead-heading is that no flow carries heat away, so the trip is *low flow on a time delay* and high pressure is only an alarm.

8.2.3 Hysteresis

A comparator that switches exactly at a limit chatters when the signal rests on it, and an alarm list that chatters is one operators stop reading. Each alarm is a latching comparator: a high alarm sets when $x \geq L$ and clears only when $x \leq L - \delta$, with deadbands $\delta = 10$ mm for levels, 2 kPa for pressure and 0.5 L/min for flow.

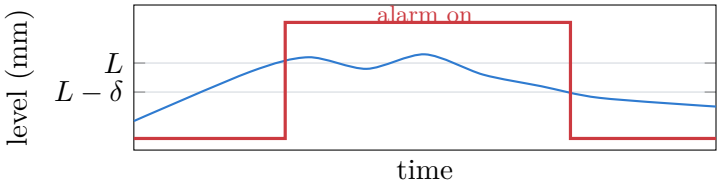


Figure 8.3: Hysteresis: the alarm (red) sets at L and clears only below $L - \delta$, so the wobble between 496 and 503 mm produces a single alarm.

8.2.4 Timers count scans

The minimum-flow trip uses an on-delay timer (TON): its output goes true once its input has been true for the preset, and resets the moment the input goes false. The timer counts *scans*, not seconds. Accumulating `elapsed += 0.1` reaches 0.9999999999999999 after ten scans, and a timer compared with `>=` against a 1.0 s preset would never fire. Real PLC timers count ticks for exactly this reason. The preset is converted once to round(T/T_s) scans.

8.3 Drift-free scheduling

`await asyncio.sleep(period)` in a loop schedules the next wake relative to when the previous one *finished*, so every microsecond of work is added permanently to the phase. The `Ticker` in `pyplant/clock.py` computes absolute targets from a fixed origin instead:

$$t_n = t_0 + n\,\frac{T}{\text{speed}}. \tag{8.1}$$

Work eats into the slack, never into the phase. With a 70 ms body on a 100 ms period, 100 ticks land at exactly 10.000 s rather than 17.000 s; a test asserts this to 10^{-12} s.

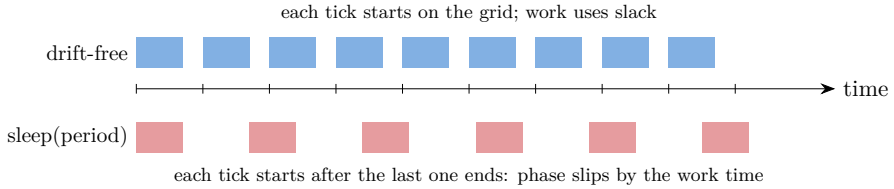


Figure 8.4: Absolute scheduling (top) against relative sleeping (bottom) with a 70% duty cycle.

8.3.1 The engine measures itself

SCAN_MS, SCAN_OVERRUNS and PLC_HEARTBEAT are real tags, trended like any process value. The published statistics are always the previous scan's, because this scan's duration is unknown until its write completes, which is what real controllers do. When the engine cannot keep up, lateness grows without bound against the fixed grid while the overrun count increments once per scan; lateness is the signal to watch.

8.4 The tag dictionary

`pyplant/tags.py` is the single source of truth for every signal. Each `Tag` carries a description, kind (analog or bool), owning process, Modbus table and address, engineering unit and span, integer scale factor, alarm limits, writability and the OPC UA area. Every other module imports its I/O definitions from it; nothing else may hard-code an address or a scale factor. Appendix A.1 lists all tags.

8.4.1 Wire representation

Analog values travel in 16-bit Modbus holding registers as

$$w = \mathrm{round}(v \cdot s), \qquad w \in [-32768, 32767], \tag{8.2}$$

sent as an unsigned two's-complement word. The scale s is chosen per tag so the span fits; this is checked at import. Out-of-range values are clamped, not wrapped, so a bad value reads as a pegged instrument rather than a sign flip. For LT101, $s = 10$: 432.7 mm travels as 4327 with a resolution of 0.1 mm. Booleans travel in coils as 0 or 1.

8.4.2 Ownership and blocks

Each tag's `owner` (`PLANT`, `PLC`, `OPER`, `ML`) is the only process that may write it. Addresses are grouped into declared *blocks* so a client polls a whole span in one Modbus transaction, and import fails if a block has a gap, two blocks overlap, or a tag sits in a block owned by another process.

No block may be written by two parties

If one block held both the operator's setpoint and the PLC's readback, an operator write arriving between the PLC's read and its publish would be overwritten 100 ms later. The faceplate is therefore split into `OPER_SP`/`OPER_CMD` (HMI writes, PLC reads) and `PLC_FACE`/`PLC_DIAG`/`PLC_ALARMS` (PLC writes, HMI reads).

8.5 Modbus TCP: the field layer

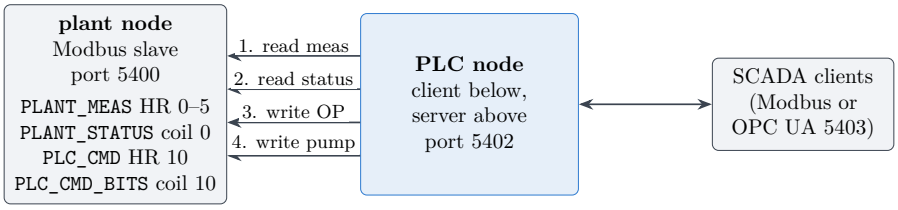


Figure 8.5: Four Modbus round trips per scan, measured at 0.7 ms against a 100 ms budget.

The plant is a Modbus *slave*, as a field device is. It has no controller, no alarms and no memory of intent: it applies whatever is in its command registers, so if the PLC stops writing, the

valve holds its last position, as a real one does. The PLC is both client (towards the plant) and server (towards SCADA). It mirrors the plant measurements into its own store so the HMI polls one address instead of two. Reads are by block, never by tag: one `read_holding_registers` for six measurements is one round trip.

8.6 OPC UA: the information layer

OPC UA is the boundary between registers and meaning. Below it everything is addresses and scale factors; above it the historian and HMI speak tag names with units attached and never import a Modbus symbol.

The address space is generated from the tag dictionary. Each tag's `area` becomes its folder path, e.g. `Plant/Unit1/LT101`, `PLC/LIC101/LIC101_SP` (writable) or `PLC/Diagnostics/SCAN_MS`. Every analog node carries `EngineeringUnits` and `EURange` from its declared span, so a client can scale a trend without being told anything about the plant.

The server is a *view* of the same `TagStore` the Modbus server holds, not a second process image. Values are pushed outward on a refresh tick only for nodes that changed; client writes are pushed back into the store. To stop a refresh from reverting a fresh operator write, `refresh()` tracks the last value each writable tag was seen to hold and overwrites a writable node only when the store changed underneath it.

Clients use *subscriptions*: the server pushes changes at a publishing interval, which is why the historian is not a poller.

Chapter 9

SCADA: historian, HMI and alarms

Supervisory control and data acquisition (SCADA) sits above the PLC. It does not close loops; it records, displays and lets people intervene. In pyplant the SCADA layer is three pieces: the historian collector, its query API and the operator HMI with an ISA-18.2 alarm manager.

9.1 The historian

9.1.1 Data flow

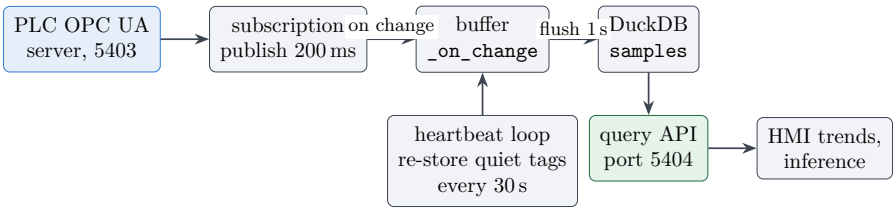


Figure 9.1: The historian collector (`pyplant/services/historian.py`).

The collector subscribes to every tag the PLC serves. An OPC UA subscription already pushes only on change, which is most of what a historian's compression does. What it does not give is an anchor for a tag that has been quiet for an hour, so each tag is also re-stored if it has been silent for `heartbeat` seconds (default 30). This is the classic exception-reporting pair: store on change, and never let the gap exceed a maximum. Writes are buffered and flushed in batches every second, because row-at-a-time inserts are the usual way to make DuckDB look slow.

9.1.2 Storage

The table is in long format, one row per sample:

```
samples(ts TIMESTAMP, tag VARCHAR, value DOUBLE, quality UTINYINT)
```

A wide table needs a migration the day a tag is added; a long table does not. Quality is the OPC UA status flattened to `GOOD` (0), `UNCERTAIN` (1) and `BAD` (2). Queries ignore bad samples unless asked.

9.1.3 Resampling: last value carried forward

Samples arrive by exception, so between two stored samples the signal did not drift; it was *held*. Linear interpolation would invent a ramp that never existed, and on a valve position or an alarm

bit that is simply wrong. The historian builds a regular grid and uses DuckDB's `ASOF LEFT JOIN` to take, for every grid instant, the last sample at or before it:

$$\hat{x}(t_g) = x(t_j), \qquad j = \max\{\, i : t_i \le t_g \,\}. \tag{9.1}$$

Samples stored before the window's start count too, so a window never opens with a column of nulls just because a tag happened to be quiet.

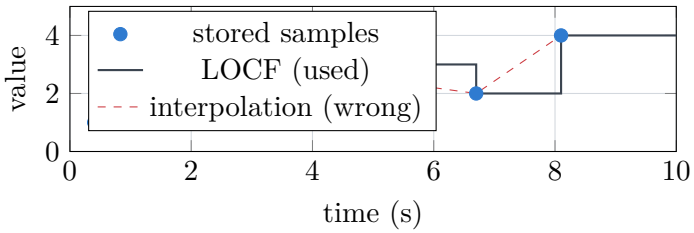


Figure 9.2: Step-hold resampling against linear interpolation.

9.1.4 Covered span

`span()` is the extent of the whole database, and a window opened there almost always starts with nulls: tags report a few milliseconds apart, so the earliest sample belongs to exactly one of them. `covered_span(tags)` returns the interval over which *every* requested tag has a value, the latest of the per-tag first samples to the latest last sample. Use it to ask "give me everything you have".

Wall-clock timestamps

Timestamps are wall clock, as a real historian's are. At `speed` > 1 the recorded timeline is compressed, so a window 60 wall seconds long covers 60 $\times$ speed seconds of plant time. Training data comes from the fast runner in plant time and never touches this database.

9.1.5 One writer

DuckDB takes an exclusive lock on its file, so a second process cannot open it even to read. That is not a limitation to work around: a historian is a service you query, not a file you share. Exactly one process writes, and everything else asks the query API on port 5404 (Chapter 16).

9.2 The operator HMI

The HMI (`pyplant/hmi/app.py`; port 5405 under the workbench, 8080 when started by hand) is a FastAPI application serving one HTML page and a JSON API. It connects to the PLC over OPC UA, polls the faceplate every 0.5 s, feeds the alarm manager and reads trends from the historian API. It shows a live mimic of the two tanks, the LIC101 faceplate (SP, PV, OP, mode), pump control, trends and the alarm list.

9.2.1 Write guards

Every operator write goes through one function that rejects bad requests before anything reaches the control system:

Check	HTTP status	Example
tag exists	404	unknown tag
tag is operator-writable	403	writing LT101
analog value inside span	422	setpoint of 900 mm
connected to the PLC	503	PLC is down

9.2.2 Pulsed reset

`POST /api/reset` pulses CMD_ALM_RESET. A latched interlock needs an *edge*, not a level: holding the reset bit true would re-clear the latch on every scan and defeat it.

9.3 Alarm management (ISA-18.2)

A boolean that turns a row red is not alarm management. ANSI/ISA-18.2 exists because operators need to distinguish four situations that a single "is it on" flag collapses into two. pyplant implements the core state model in `pyplant/hmi/alarms.py`.

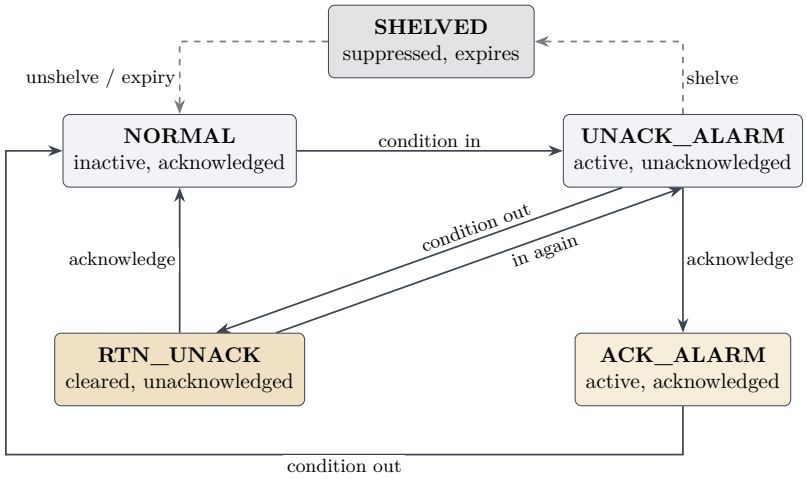


Figure 9.3: The ISA-18.2 alarm states implemented by `Alarm`. Shelving can be entered from any state and returns to whatever the condition says at that moment.

NORMAL quiet, and nobody needs to do anything.

UNACK_ALARM active, and nobody has looked at it yet.

ACK_ALARM active, and somebody has taken responsibility.

RTN_UNACK it cleared itself before anyone looked. This is the state that matters most: a transient between two operator glances leaves no trace in a flag-based list, and the transient nobody saw is often the interesting one. It stays on the list until acknowledged.

SHELVED a deliberate, temporary, expiring silence with a reason recorded (default 30 min, capped at 8 h). The condition is still tracked underneath: if it goes active while shelved, the alarm comes back as unacknowledged on expiry.

9.3.1 Priorities

Priority is a design decision per alarm, not something derived from a name. An interlock that trips the pump outranks a level approaching its limit.

Alarm	Priority	Message
ILK_PMP101	CRITICAL (4)	interlock tripped, pump stopped, valve shut
ALM_LT101_LOLO	HIGH (3)	tank 1 level low low
ALM_PT101_HIHI	HIGH (3)	pump discharge pressure high high
ALM_LT101_HI	MEDIUM (2)	tank 1 level high
ALM_HEALTH	LOW (1)	model health index degraded

The alarm page sorts by priority, then newest first. The summary reports the counts per state, the number needing attention (UNACK plus RTN_UNACK) and the worst priority among them, which drives the banner colour.

9.3.2 Operating the alarm list

1. A red row is an unacknowledged active alarm. Investigate, then press **Ack**; it turns amber (acknowledged) and stays until the condition clears.
2. A row marked *returned* cleared on its own. Read it and acknowledge it; it then leaves the list.
3. A chattering nuisance alarm can be shelved with a reason and a duration. Shelving is how you avoid operators ignoring the whole list instead.
4. The interlock additionally needs **Reset** once the cause is gone; acknowledging only clears the notification, not the trip.

Chapter 10

Faults and scenarios

Faults are objects applied to a running plant, never edits to the physics. This chapter describes the fault taxonomy in `pyplant/faults/model.py`, the scenarios in `pyplant/faults/scenarios.py` that turn faults into labelled data, and how to inject faults into the live stack.

10.1 Where a fault enters

There are three places a fault can enter the loop, and which one it uses is the most important thing about it.

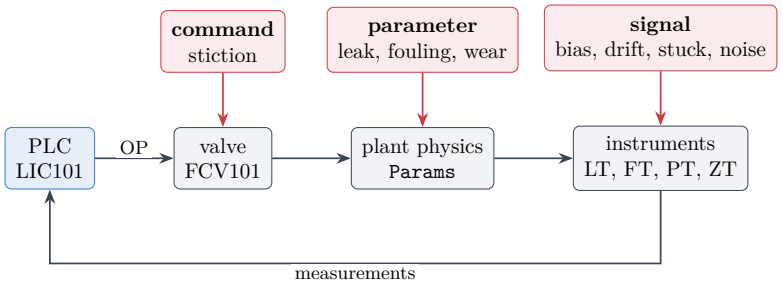


Figure 10.1: The three fault entry points, applied by `FaultSet.command()`, `FaultSet.params()` and `FaultSet.measure()`.

Parameter faults change the plant itself: the physics really is different afterwards.

Command faults sit between the PLC and the valve: the stem does not do what it is told.

Signal faults corrupt what an instrument reports while the process is unchanged.

Why the entry point matters

A level transmitter reading 30 mm high and a genuine 30 mm rise produce identical numbers in LT101. They differ only in what the *rest* of the plant does: a real rise changes LT102 and the outflow, a bias does not. A model that has never seen both cannot tell them apart. In a closed loop it is worse: the controller acts on the biased reading and moves the *true* level 25 mm down while the displayed level sits on setpoint.

10.2 Timing

Every fault has a start t_s, an optional end t_e and a ramp T_r. Its strength at time t is

$$s(t) = \begin{cases} 0 & t < t_s \text{ or } t \geq t_e, \\ \min\big((t - t_s)/T_r,\ 1\big) & T_r > 0, \\ 1 & \text{otherwise.} \end{cases} \tag{10.1}$$

The ramp is not cosmetic: an abrupt leak and an incipient one over an hour are different detection problems, and a dataset containing only step changes trains a detector that is useless on faults that creep.

10.3 The fault catalogue

Kind	Entry	Magnitude (default)	Effect
`leak`	parameter	`cda` $2 \times 10^{-5}\,\mathrm{m}^2$	adds a Torricelli drain on tank 1: $q_{\text{leak}} = C_d A\sqrt{2gh_1}$
`fouling`	parameter	`loss` 0.30	interconnect C_dA multiplied by $(1 - \text{loss} \cdot s)$
`pump_wear`	parameter	`loss` 0.15	pump head multiplied by $(1 - \text{loss} \cdot s)$
`stiction`	command	`deadband` 0.06, `slip` 0.04	stem sticks inside S, jumps by J
`sensor_bias`	signal	`bias` 25 (units)	reading $+\,\text{bias} \cdot s$
`sensor_drift`	signal	`rate_per_min` 5	reading $+\,\text{rate} \cdot (t - t_s)/60$
`sensor_stuck`	signal	none	reading frozen at onset value
`sensor_noise`	signal	`sigma` 1.0	reading $+\,\mathcal{N}(0, (\sigma s)^2)$, seeded

Signal faults take a `tag` (default LT101); the corrupted value is clamped to the tag's span, so a bias cannot read beyond the instrument's range. Several faults can be active together; the label of a sample is the sorted list of active fault ids joined by `+`, or `normal`.

10.4 Valve stiction in detail

Stiction is the most control-relevant fault in the set and the hardest one for a detector fed only LT101, because its signature lives in the phase relationship between FCV101_OP and ZT101. pyplant implements the two-parameter data-driven model (deadband S plus slip jump J):

1. While stuck, the stem position y does not move.
2. When the command u moves more than S away from y, the stem breaks free and lands at $u - \text{sign}(u - y)(S - J)$, i.e. it jumps by J past the point it would have reached smoothly.
3. While moving, the stem follows the command.
4. On a reversal of the command direction, it sticks again.

In a closed loop with integral action the result is a *limit cycle*: the controller keeps winding until the stem slips, the level overshoots, the controller winds back the other way, the stem sticks on the reversal, and the pattern repeats. The parameters are fractions of full stroke and the slip cannot exceed the deadband.

10.5 Instrument noise is not a fault

Every scenario adds Gaussian noise to the transmitters:

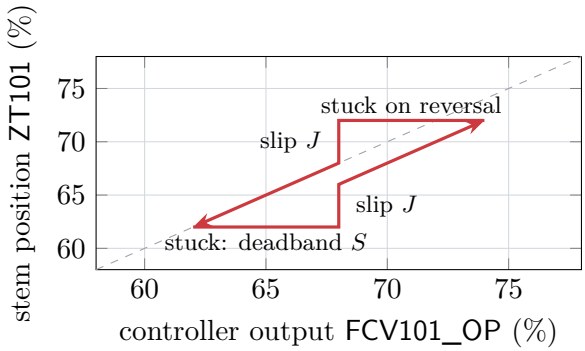


Figure 10.2: Idealised stiction cycle in the OP–stem plane. A healthy valve lies on the dashed diagonal; a sticky one traces a parallelogram.

Tag	LT101	LT102	FT101	PT101	ZT101
σ	0.4 mm	0.4 mm	0.03 L/min	0.08 kPa	0.05 %

It is applied as *benign* `SensorNoise` faults: recorded, never labelled positive. This is not decoration. A perfectly quiet loop never moves its valve, so stiction cannot produce a limit cycle at all, and a detector trained on noiseless signals learns a threshold that collapses the moment it meets a real transmitter.

10.6 Scenarios

A `Scenario` is a reproducible run: a duration, a setpoint programme, a fault timeline, a seed, plant parameters and a noise table. Running it produces a trajectory in which every sample is labelled.

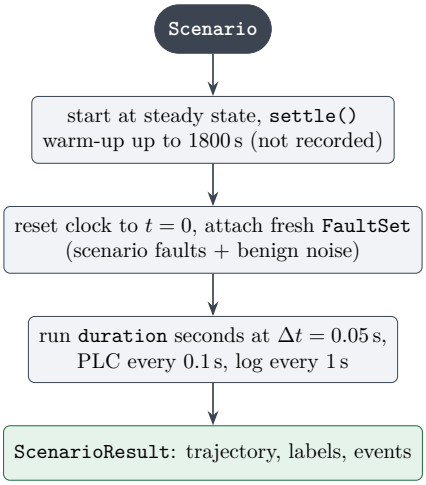


Figure 10.3: `run_scenario()`. The warm-up stops a model from learning that "faults happen at the start".

Fault objects are stateful (stiction carries a stem position), so a fresh copy is built for every run and the scenario's own fault list is never mutated.

10.6.1 The library

`library(duration)` returns one representative scenario per fault mode plus normal operation, each with onset at 40% of the duration and the setpoint programme 300, 340, 280, 300 mm at 0,

900, 1800, 2700 s:

Name	Description
`normal`	setpoint moves only, no fault
`leak_abrupt`	tank 1 leak, step onset
`leak_incipient`	tank 1 leak opening over fifteen minutes
`pump_wear`	impeller wear 18%, 600 s ramp
`fouling`	interconnect restriction 35%, 1200 s ramp
`sensor_bias`	LT101 reads 25 mm high
`sensor_drift`	LT101 drifting up at 1.5 mm/min
`sensor_stuck`	LT101 frozen
`stiction`	FCV101 sticking, deadband 6%, slip 4%

10.6.2 The grid

`grid(duration, magnitudes, repeats)` sweeps each fault mode across `magnitudes` scale factors from 0.5 to 1.5, repeats each with different seeds, and draws a random onset between 25% and 60% of the duration. It also adds `repeats` normal runs. A model therefore cannot learn "the fault starts at 24 minutes" instead of learning the fault. The number of scenarios is

$$N = \text{repeats}\,(1 + 8 \times \text{magnitudes}), \tag{10.2}$$

so the Standard preset (3 magnitudes, 2 repeats) builds 50 scenarios. `default_dataset()` then tops the normal runs up to `normal_runs` (10 for Standard), giving 58 runs in all.

10.7 Injecting faults into the live stack

The plant service exposes an injection API (Chapter 16); the workbench **Faults** tab is a form over it.

1. Start the stack from **Runtime** and wait for all services to be up.
2. On **Faults**, choose a kind, a magnitude, a ramp and, for sensor faults, a tag. Times are in plant seconds from now.
3. Press **Inject**. The fault appears in the active list with its id.
4. Watch the HMI: the PV may not move at all (integral action), but the OP and the health index will.
5. Clear one fault or all of them. `GET /api/truth` always returns the true state, including what the instruments do not show.

From Python:

```
from pyplant.faults.model import Leak
from pyplant.faults.scenarios import Scenario, run_scenario
sc = Scenario("my_leak", duration=1800,
              faults=[Leak("leak", t_start=600, cda=3e-5, ramp=300)])
res = run_scenario(sc)
df = res.frame() # time series + label + faulty columns
print(res.events) # ground truth
```

Chapter 11

Machine learning for fault detection

This chapter covers the data-driven diagnosis pipeline in `pyplant/ml`: how runs become feature windows, the theory behind each classical detector, how thresholds and the health index are set, and how a detection is traced back to an instrument. The neural detector is in Chapter 12 and the evaluation method in Chapter 13.

11.1 The pipeline

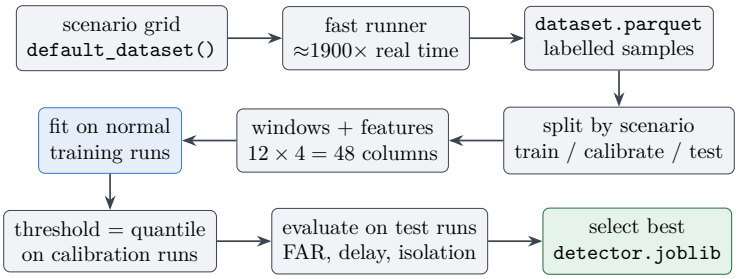


Figure 11.1: From simulation to a deployable detector. The workbench **Data** and **Model** tabs run exactly these steps.

11.2 Why unsupervised, trained on normal only

The detectors never see a fault during training. This is not a purity exercise: a real plant has years of normal operation and a handful of documented failures, so a supervised classifier would be trained on the rarest thing it will ever meet and could not recognise a mode nobody labelled. Instead each detector learns what *normal* looks like and scores how far a window departs from it. Fault data is used only to measure the result.

Formally, given normal feature vectors $\{\boldsymbol{x}_i\}$ drawn from p_0, each detector defines an anomaly score $a(\boldsymbol{x})$ that is small where p_0 is dense and large where it is not. A window is flagged when $a(\boldsymbol{x}) > \theta$.

11.3 Windows

A detector looks at a window, not a sample, because what distinguishes faults are shapes over time: a leak shifts a level, a drift tilts it, stiction makes it oscillate. One sample cannot express any of those.

Defaults: `WINDOW = 60` samples (one minute) and `STRIDE = 10`. A window labelled by its first sample would make detection delay look negative.

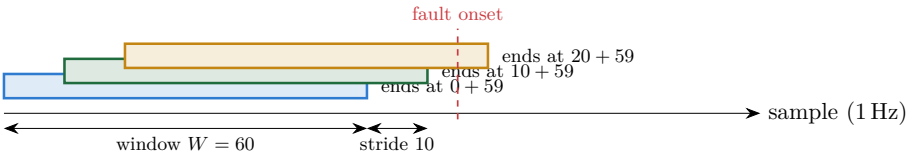


Figure 11.2: Overlapping windows. Each is labelled by its *last* sample, so a window straddling the onset is not counted as faulty from its first sample.

11.4 Features

11.4.1 Channels

Nine measured or controller tags are used: LT101, LT102, FT101, FT102, PT101, ZT101, FCV101_OP, LIC101_SP and LIC101_ERR. Controller output is included deliberately: in a closed loop the controller absorbs the fault, so the PV alone hides it (Section 7.5).

Three more channels are computed from first principles, each because the fault shows up in a difference that no single tag contains:

$$\text{OP_MINUS_ZT} = \text{FCV101_OP} - \text{ZT101}, \tag{11.1}$$

$$\text{FLOW_IMBALANCE} = \text{FT101} - \text{FT102}, \tag{11.2}$$

$$\text{HEAD_RESIDUAL} = \text{PT101} - \frac{\Delta p_0}{1000}\left(1 - \left(\tfrac{q}{q_{\max}}\right)^2\right), \tag{11.3}$$

where q is FT101 in m^3/s. OP_MINUS_ZT is zero on a healthy valve, and a sticking one opens a gap. FLOW_IMBALANCE is zero at steady state by conservation of mass, and a leak makes it positive: a conservation law, not a learned correlation. HEAD_RESIDUAL is the pump pressure minus what the pump curve says it should develop at the delivered flow, and impeller wear opens it. Giving a model the residual of a physical relationship is cheaper and more robust than making it learn the relationship.

11.4.2 Statistics

Each of the 12 channels is reduced to four numbers over the window, $\{last, mean, std, slope\}$, giving 48 features. The slope is the least-squares slope in units per sample,

$$\beta = \frac{\sum_k (k - \bar{k})(x_k - \bar{x})}{\sum_k (k - \bar{k})^2}. \tag{11.4}$$

Statistic	Meaning	Where faults show
last	where it is now	bias, stuck
mean	where it has been	leak, fouling, wear
std	how much it is moving	stiction (limit cycle), stuck (std $\to$ 0)
slope	which way it is going	drift, incipient faults

Sixty samples of twelve channels would be 720 nearly redundant numbers. Four statistics per channel is small enough to train on a modest dataset and keeps every feature attributable to one channel, which is what makes isolation possible. Features are named `CHANNEL::stat`, e.g. `FLOW_IMBALANCE::mean`.

11.4.3 Standardisation

Every detector first standardises with statistics of the normal training set:

$$z_j = \frac{x_j - \mu_j}{\sigma_j}. \tag{11.5}$$

In these units "normal" is centred at the origin with unit spread in every direction, and all scores below are computed on $\boldsymbol{z}$.

11.5 Detector 1: z-score limits

The most straightforward method there is: no model at all. The reconstruction is the normal mean, which is zero in standardised units, so the score is

$$a_{\mathrm{z}}(\boldsymbol{x}) = \sum_{j=1}^{48} z_j^2. \tag{11.6}$$

Interpretation

If the features were independent Gaussians, a_{z} would follow a χ^2 distribution with 48 degrees of freedom and a fixed limit would set a known false alarm rate. They are not independent (LT101 and LIC101_ERR are nearly the same signal), which is why the threshold is taken empirically from calibration data instead. The z-score is the baseline every other method must beat; if it cannot be beaten, the honest report says so.

11.6 Detector 2: PCA and the SPE statistic

Principal component analysis is the basis of classical multivariate statistical process control (MSPC). Normal operation of a plant governed by a few physical relationships occupies a low-dimensional subspace of feature space; a fault that breaks a relationship pushes data *off* that subspace.

11.6.1 Theory

With the standardised training matrix $Z \in \mathbb{R}^{n \times m}$, the covariance $S = Z^{\top}Z/(n-1)$ has the eigendecomposition $S = P\Lambda P^{\top}$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots$. Keep the first k eigenvectors P_k, chosen as the smallest k that explains 95% of the variance:

$$\frac{\sum_{i \leq k} \lambda_i}{\sum_i \lambda_i} \geq 0.95. \tag{11.7}$$

A window is projected onto the subspace and back, $\hat{\boldsymbol{z}} = P_k P_k^{\top} \boldsymbol{z}$, and the score is the *squared prediction error* (SPE, the Q statistic):

$$a_{\mathrm{PCA}}(\boldsymbol{x}) = \mathrm{SPE} = \|\boldsymbol{z} - \hat{\boldsymbol{z}}\|^2 = \|(I - P_k P_k^{\top})\,\boldsymbol{z}\|^2. \tag{11.8}$$

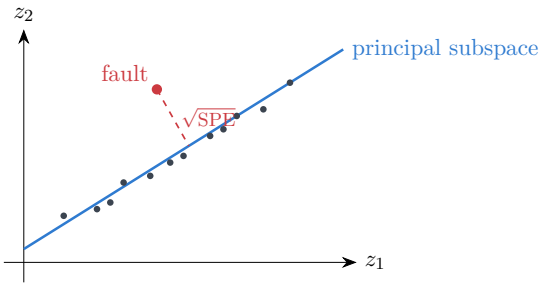


Figure 11.3: Normal data (grey) lies near the principal subspace. A fault that breaks the correlation is far from it even when each coordinate is within its normal range.

The second classical statistic, Hotelling's $T^2 = \boldsymbol{z}^{\top} P_k \Lambda_k^{-1} P_k^{\top} \boldsymbol{z}$, measures unusual variation *within* the subspace. pyplant uses SPE only, because the faults of interest break correlations rather than scale them.

11.7 Detector 3: isolation forest

An isolation forest does not model density or reconstruct anything. It exploits the observation that anomalies are *few and different*, so random partitioning isolates them in fewer splits than normal points.

11.7.1 Theory

Each of the $t = 200$ trees is built on a random subsample of size ψ by repeatedly choosing a random feature and a random split value between its minimum and maximum, until every point is alone. The path length $h(\boldsymbol{x})$ is the number of splits needed to isolate $\boldsymbol{x}$. Averaged over trees and normalised by the expected path length of an unsuccessful search in a binary search tree,

$$c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{\psi}, \qquad H(i) \approx \ln i + 0.5772, \tag{11.9}$$

the anomaly score is

$$a_{\mathrm{IF}}(\boldsymbol{x}) = 2^{-\mathbb{E}[h(\boldsymbol{x})]/c(\psi)} \in (0, 1]. \tag{11.10}$$

Scores near 1 are anomalies, scores well below 0.5 are normal. The code uses the negated `score_samples` of scikit-learn, which is exactly this quantity, so larger is worse and the threshold and health index work unchanged.

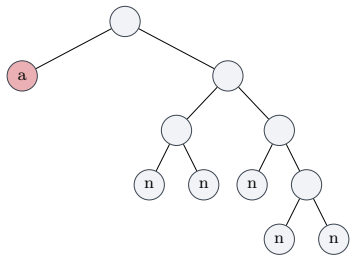


Figure 11.4: One isolation tree. The anomaly `a` is isolated after one split; normal points `n` need three or four.

Because it has no reconstruction, the isolation forest's contributions fall back to the standardised deviations z_j^2 (the same as the z-score detector).

11.8 Thresholds

Every detector's threshold is a quantile of its score on *held-out normal* data:

$$\theta = Q_q\big(\{a(\boldsymbol{x}_i) : \boldsymbol{x}_i \in \text{calibration runs}\}\big), \qquad q = 0.995. \tag{11.11}$$

Setting it on the training set measures how well the model memorised, not how often it will cry wolf. With $q = 0.995$, about one normal window in 200 is above threshold; the persistence rule of Chapter 13 turns that into near-zero false alarms.

11.9 The health index

Raw scores have no stable scale: retrain the model and they change entirely. The health index maps any score onto 0–100 with the threshold pinned at 50 and one decade of score per 50 points:

$$\mathrm{HI} = \mathrm{clip}\Big(50 \log_{10} \frac{a}{\theta} + 50,\ 0,\ 100\Big). \tag{11.12}$$

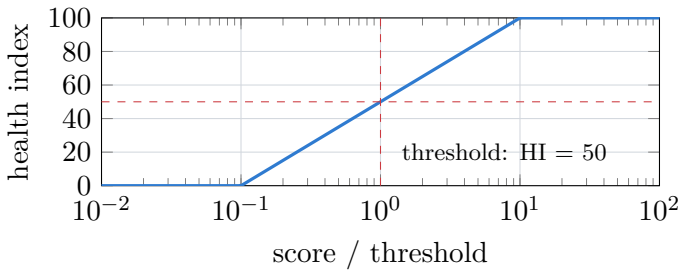


Figure 11.5: Health index against normalised score. A badly broken plant (10× threshold, HI 100) stays distinguishable from a marginal one (HI just above 50).

Pinning the threshold to 50 is what lets an alarm limit on the HEALTH_INDEX tag stay where the alarm engineer put it across a retrain.

Online inference

The tag dictionary reserves an `ML_RESULT` block (HEALTH_INDEX, TOP_CONTRIB, ML_HEARTBEAT) and the ALM_HEALTH alarm for an inference service that writes the health index back into the control system. This release computes scores, health index and contributions in Python and in the workbench **Model** tab; no service writes those tags yet. Chapter 15 sketches how to add one.

11.10 Isolation: contributions

Detection says *that* something is wrong; isolation says *where*. The reconstruction-based detectors produce a per-feature residual $\boldsymbol{r} = \boldsymbol{z} - \hat{\boldsymbol{z}}$. Every feature belongs to one channel, so summing squared residuals by channel and normalising gives each channel's share of the anomaly:

$$c_k = \frac{\sum_{j \in \text{channel } k} r_j^2}{\sum_j r_j^2}, \qquad \sum_k c_k = 1. \tag{11.13}$$

The *top channel* is $\arg\max_k c_k$. This is the same argument classical contribution plots rest on. Expected top channels for a fully developed fault:

Fault	Expected top channel
leak	FLOW_IMBALANCE
pump wear	HEAD_RESIDUAL
stiction	OP_MINUS_ZT
fouling	LT102 or FT102
sensor faults	not scored (see Chapter 13)

11.11 Using the detectors from Python

```
from pyplant.ml.dataset import load, split_scenarios, windowed
from pyplant.ml.detector import make_detector
from pyplant.ml.features import WindowSpec

frame = load("work/demo/dataset.parquet")
data = windowed(frame, WindowSpec(window=60, stride=10))
split = split_scenarios(frame, seed=0)
```

```
det = make_detector("pca", quantile=0.995) # zscore | pca | isolation_forest |
    autoencoder
det.fit(data.of_scenarios(split.train).X, data.names)
det.calibrate(data.of_scenarios(split.calibrate).X)

test = data.of_scenarios(split.test_faulty)
hi = det.health(test.X) # 0-100, 50 = threshold
top = det.top_channel(test.X) # e.g. 'FLOW_IMBALANCE'
det.save("my_detector.joblib")
```

Chapter 12

Deep learning: the autoencoder detector

PCA finds the best *linear* subspace for normal data. The plant is not linear: Torricelli drains go as $\sqrt{h}$, the pump curve is quadratic and the valve is equal-percentage. An autoencoder learns a *non-linear* manifold with the same scoring and the same contribution analysis. This chapter explains the neural-network theory behind it and how it is trained here.

12.1 Neural network fundamentals

12.1.1 The neuron and the layer

A layer maps an input vector $\boldsymbol{h}^{(l-1)}$ to

$$\boldsymbol{h}^{(l)} = \phi(W^{(l)}\boldsymbol{h}^{(l-1)} + \boldsymbol{b}^{(l)}), \tag{12.1}$$

with weights $W^{(l)}$, biases $\boldsymbol{b}^{(l)}$ and an element-wise non-linear activation ϕ. Stacking layers gives a multilayer perceptron (MLP), which by the universal approximation theorem can represent any continuous function on a compact set given enough units.

12.1.2 Activation

pyplant uses the hyperbolic tangent,

$$\tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}, \qquad \tanh'(a) = 1 - \tanh^{2}(a). \tag{12.2}$$

It is smooth, zero-centred (which suits standardised inputs) and saturates gently. The output layer is linear, because standardised features are not bounded.

12.1.3 Learning: loss, backpropagation and Adam

Training minimises the mean squared error over a batch $\mathcal{B}$,

$$\mathcal{L}(\Theta) = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}} \|\boldsymbol{z}_i - f_\Theta(\boldsymbol{z}_i)\|^2 + \alpha\|\Theta\|^2, \tag{12.3}$$

where the second term is a small L2 weight penalty ($\alpha = 10^{-4}$, the scikit-learn default). Gradients $\nabla_\Theta \mathcal{L}$ are computed by backpropagation, the chain rule applied layer by layer from the output

back to the input. Parameters are updated with Adam, which keeps running averages of the gradient and its square:

$$\boldsymbol{m}_t = \beta_1 \boldsymbol{m}_{t-1} + (1-\beta_1)\boldsymbol{g}_t, \qquad \boldsymbol{v}_t = \beta_2 \boldsymbol{v}_{t-1} + (1-\beta_2)\boldsymbol{g}_t^2, \tag{12.4}$$

$$\Theta_t = \Theta_{t-1} - \eta \frac{\hat{\boldsymbol{m}}_t}{\sqrt{\hat{\boldsymbol{v}}_t} + \epsilon}, \qquad \hat{\boldsymbol{m}}_t = \frac{\boldsymbol{m}_t}{1-\beta_1^t},\ \hat{\boldsymbol{v}}_t = \frac{\boldsymbol{v}_t}{1-\beta_2^t}, \tag{12.5}$$

with learning rate $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

12.1.4 Generalisation and early stopping

A network with enough capacity can memorise its training set, in which case it reconstructs *everything* well, faults included, and becomes useless as a detector. 15% of the training windows are held back as a validation set, and training stops when the validation loss has not improved for 25 epochs (*early stopping*), up to a maximum of 1500 iterations.

12.2 The autoencoder

An autoencoder is a network trained to reproduce its own input through a narrow *bottleneck*. The encoder compresses the 48 standardised features to 8 numbers; the decoder expands them back. Because the bottleneck is narrower than the input, the network can reproduce only inputs that resemble what it was trained on: it learns the manifold of normal operation.

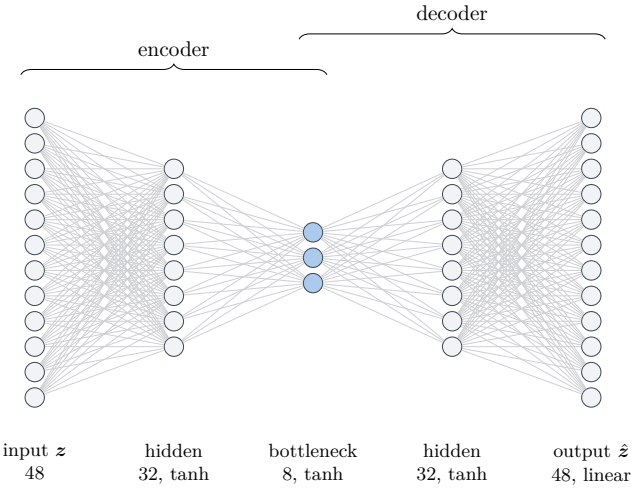


Figure 12.1: The 48–32–8–32–48 autoencoder (drawn with fewer units). It is trained to output its input.

Setting	Value	Field in `AutoencoderDetector`
hidden layers	(32, 8, 32)	`hidden`
activation	tanh	fixed
optimiser	Adam, $\eta = 10^{-3}$	fixed
maximum iterations	1500	`max_iter`
early stopping	15% validation, patience 25	fixed
random seed	0	`seed`

12.2.1 Relationship to PCA

A linear autoencoder (identity activations) trained with MSE learns exactly the principal subspace: its optimum spans the same space as the top k principal components. PCA is therefore the linear

special case, and the autoencoder earns its place only if the non-linearity buys detection that PCA misses. That is why both are trained and the comparison is kept on record.

12.3 Scoring, threshold and isolation

The autoencoder uses the same machinery as PCA:

$$a_{\mathrm{AE}}(\boldsymbol{x}) = \|\boldsymbol{z} - f_\Theta(\boldsymbol{z})\|^2, \tag{12.6}$$

the same calibration quantile, the same health index and the same per-channel contributions of the residual (Chapter 11). Only the reconstruction changes.

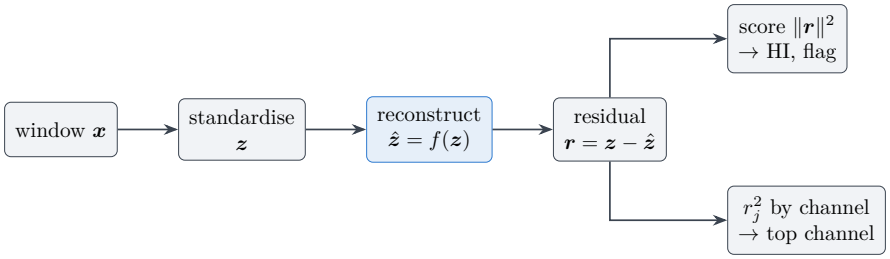


Figure 12.2: One scoring pass, identical for PCA and the autoencoder.

12.4 Practical advice

- **More data before more layers.** With the Quick preset there are only a few normal runs; a bigger network will memorise them. Use the Standard or Thorough preset before changing `hidden`.
- **Bottleneck width** controls the trade-off: too wide and faults reconstruct well (missed detections), too narrow and normal variation does not (false alarms). 8 is a good starting point for 48 features.
- **Seeds matter.** Training is stochastic. Compare methods over two or three seeds before concluding one is better.
- **Check against PCA.** If the autoencoder does not beat PCA on detection rate and false alarms, deploy PCA. The selection rule does this automatically.

To change the architecture:

```
from pyplant.ml.detector import AutoencoderDetector
det = AutoencoderDetector(quantile=0.995, hidden=(64, 16, 64), max_iter=3000, seed=1)
```

Chapter 13

Evaluating and selecting a detector

A detector's numbers mean nothing unless the data split is honest and the metrics are the ones an operator would recognise. This chapter describes the split in `pyplant/ml/dataset.py`, the metrics in `pyplant/ml/evaluate.py`, and the rule that picks the detector to deploy.

13.1 Split by scenario, never by sample

Windows overlap, and neighbouring windows from one run are nearly identical. A random sample split puts window 41 in training and window 42 in test; the model scores brilliantly on data it has effectively already seen, and the number is not slightly optimistic, it is meaningless. Whole runs are split instead.

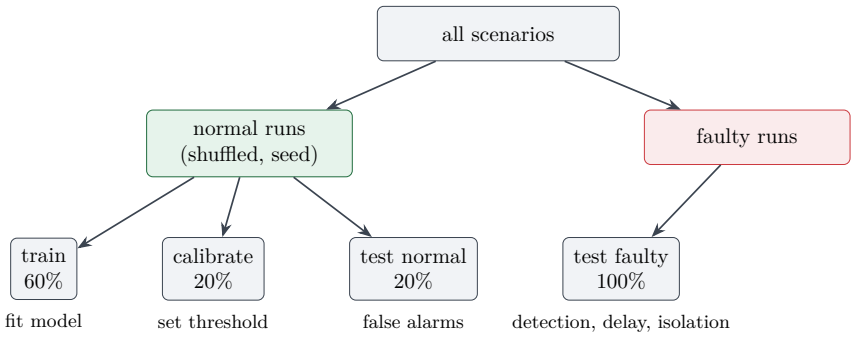


Figure 13.1: `split_scenarios()`. Normal runs are divided three ways; faulty runs are used only for testing.

Each pool has a single job. Using one pool for fitting, thresholding and false alarm measurement flatters the threshold. At least three normal runs are required; the Quick preset makes six and the Standard preset ten. Because each grid entry has its own seed, magnitude and onset, held-out runs differ from training runs in the ways that matter operationally.

13.2 Persistence

A single flagged window is not a detection. A detection is declared at the first window of the first run of $p = 3$ consecutive flagged windows (`PERSISTENCE`). With a 10 s stride this costs at most 20 s of delay.

A detector that flickers above threshold one window in twenty has a fine-looking detection rate and is unusable. The same rule counts false alarms, so both numbers describe the same detector.

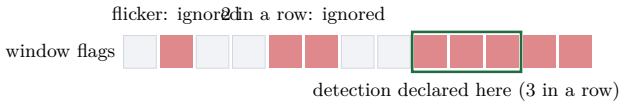


Figure 13.2: The persistence rule. Isolated flags never count, either as detections or as false alarms.

13.3 The metrics

13.3.1 False alarms per day

Measured on the held-out normal runs:

$$\mathrm{FAR} = \frac{\text{number of persistent excursions}}{\text{normal time covered (s)}} \times 86\,400. \tag{13.1}$$

Operators experience alarms per shift, not per sample. The report also gives the raw fraction of normal windows above threshold, which should be close to $1 - q$.

13.3.2 Detection rate and delay

For each faulty test run, the detection delay is the time from the declared onset to the first persistent detection after it:

$$\tau_d = t_{\mathrm{detect}} - t_{\mathrm{onset}}. \tag{13.2}$$

Detection rate is the fraction of faulty runs detected; the report gives the median delay overall and per fault mode. Incipient faults (long ramps) have naturally longer delays.

13.3.3 Isolation accuracy

Once a fault has fully developed (ramp finished plus `DEVELOPED_AFTER` = 300 s), the most frequent top channel over the remaining windows is compared with the channel physics names (Chapter 11). Judging earlier is wrong: fouling ramping over twenty minutes is still a transient five minutes in, and the transient implicates the controlled level rather than the downstream one.

If a run ends before the fault has developed, isolation is judged on the last quarter of the post-onset data and the run is marked as not fully developed.

13.3.4 Consistency

For each mode, the fraction of runs whose top channel is that mode's most common top channel. It needs no ground truth, so it is the only isolation number available for sensor faults.

Why sensor faults are not scored for isolation

In a closed loop, a level transmitter reading 25 mm high is driven back to setpoint by the controller: the reading looks perfect and the true level sits 25 mm low. The signature appears everywhere *except* the faulty instrument, so "the top channel should be `LT101`" is exactly backwards. Scoring against that expectation would be scoring against a mistake.

13.4 Selecting the detector

Each chosen method is fitted, calibrated and evaluated on the same split. The one deployed is

$$\arg\max_{m} \left(\text{detection rate}_m,\ -\mathrm{FAR}_m\right) \tag{13.3}$$

in lexicographic order: more detections wins, a tie goes to fewer false alarms, and remaining ties go to the simpler model because the methods are tried simplest first (z-score, PCA, isolation forest, autoencoder). A learned detector that cannot beat PCA is not deployed just because it is newer.

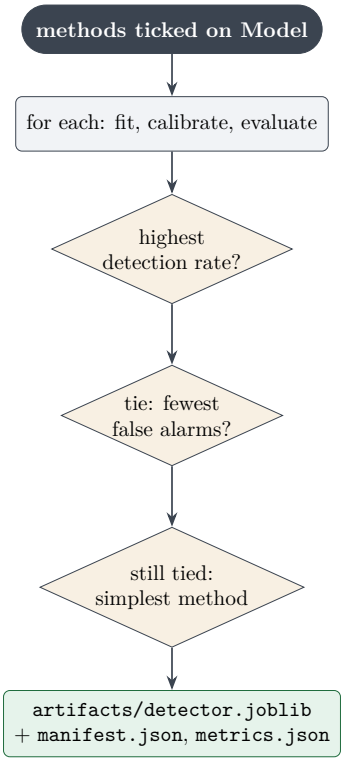


Figure 13.3: Model selection in `train_detector()`.

13.5 Reference results

With the full dataset (58 one-hour runs: `python -m pyplant.ml.train -duration 3600`) the project's reference numbers are:

	Detection	Median delay	False alarms	Isolation
PCA (MSPC baseline)	83.3%	256 s	0 / day	60%
Autoencoder	**100%**	**70 s**	**0 / day**	**73%**

PCA detects only a third of the sensor bias and drift runs; the autoencoder detects all of them. The physics channels do the isolation: leak runs point at FLOW_IMBALANCE 12 out of 12 times, pump wear at HEAD_RESIDUAL 6/6 and stiction at OP_MINUS_ZT 6/6. The raw window flag rate on normal runs is 0.14% and there are no persistent excursions. The z-score and isolation forest detectors were added later; run them on your own dataset to place them in this table. Smaller presets give noisier numbers.

13.6 Reading the Model tab

1. The comparison table has one row per method: detection rate, median delay, false alarms per day, flagged normal windows and isolation. The deployed method is marked *(selected)*.

2. The per-mode table shows, for the selected method, the number of runs of each fault mode, the fraction detected, the median delay, the most common top channel and the channel physics expects.
3. A low detection rate on one mode with a long delay usually means the fault was still ramping when the run ended; use longer runs.
4. A non-zero false alarm rate means the threshold is too low for this data: raise the quantile (e.g. 0.999) or generate more normal runs.

Chapter 14

Changing parameters

Parameters live in four places: the workbench forms, the workspace `config.json`, the command-line flags of each service, and constants in the code. This chapter says which is which, what each parameter does, and how to change it safely.

14.1 Where a parameter takes effect

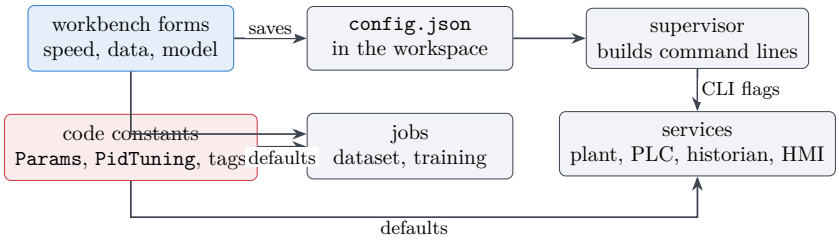


Figure 14.1: How a parameter reaches the code that uses it.

What is and is not wired

The supervisor passes only `dt`, `speed`, `loop.setpoint` and `loop.scan_period` from `config.json` to the services. The `plant` section and `loop.kc`, `ti`, `td`, `tf` are saved and read back, but the live services always use the code defaults of `Params` and `PidTuning`. To experiment with plant parameters or tuning, use the offline harness (Section 14.5) or change the defaults in the code.

14.2 The workspace configuration

`config.json` is created on first start and rewritten atomically (to a temporary file, then moved into place) whenever the workbench saves.

```
{
  "name": "demo",
  "speed": 1.0, // real-time factor of the live stack
  "dt": 0.05, // s, plant integration step
  "ports": { "plant_modbus": 5400, "plant_control": 5401, "plc_modbus": 5402,
             "plc_opcua": 5403, "historian_api": 5404, "hmi": 5405 },
  "plant": { "area_tank1": 0.02, "area_tank2": 0.02, "height_max": 0.6,
             "pump_dp0": 60000.0, "pump_qmax": 0.000417,
             "valve_cv": 2.429e-06, "valve_rangeability": 30.0,
             "tau_valve": 2.0, "cda_12": 6.9e-05, "cda_2out": 9e-05 },
```

```
  "loop": { "kc": 0.35, "ti": 72.0, "td": 0.0, "tf": 5.0,
            "setpoint": 300.0, "scan_period": 0.1 },
  "data": { "duration": 3600.0, "magnitudes": 3, "repeats": 2,
            "normal_runs": 10, "window": 60, "stride": 10,
            "quantile": 0.995, "seed": 0 }
}
```

(JSON has no comments; they are added here for explanation.) Edit it with the workbench stopped, or use the forms. A missing key falls back to its default.

14.2.1 Runtime parameters

Parameter	Default	Set from	Effect
`speed`	1.0	**Runtime** menu	real-time factor of plant and PLC (1, 2, 4, 10 or 20); saved when the stack starts
`dt`	0.05 s	`config.json`	RK4 integration step of the plant
`loop.setpoint`	300 mm	`config.json`	initial LIC101 setpoint; the operator can change it live
`loop.scan_period`	0.10 s	`config.json`	PLC scan period T_s
`ports`	base 5400	`-base-port`	six consecutive ports from the base

Speed on Windows

At `speed` s the PLC wakes every T_s/s wall seconds. Windows timers have a granularity of about 15.6 ms, so above 4× the scan period (25 ms at 4×) approaches the timer resolution and scans overrun. Use $\leq 4\times$ on Windows; slow faults such as stiction need a few process time constants (72 s each) to develop.

14.2.2 Dataset parameters (Data tab)

Parameter	Meaning	Quick	Standard	Thorough
`duration`	seconds per run	1200	3600	3600
`magnitudes`	scale factors per fault	1	3	5
`repeats`	seeds per magnitude	1	2	3
`normal_runs`	minimum normal runs (≥ 3)	6	10	16
runs generated		14	58	136
approximate time		1.5 min	4 min	10 min

Guidance: more magnitudes give the detector a wider view of fault severity; more repeats make the evaluation numbers less noisy; more normal runs give a better threshold and a more trustworthy false alarm rate. Duration must leave time after onset (25–60% of the run) for ramped faults to develop.

14.2.3 Model parameters (Model tab)

Parameter	Default	Effect
methods	all four	which detectors to fit and compare
`window`	60 samples	longer windows see slower faults and smooth noise, at the cost of delay
`stride`	10 samples	spacing of windows; smaller means finer delay resolution and more data
`quantile`	0.995	threshold quantile on calibration data; raise it (0.999) for fewer false alarms, lower it for faster detection
`seed`	0	split and model randomness (Python only)

14.2.4 Fault parameters (Faults tab)

Kind	Starting magnitude	Unit
`leak`	5×10^{-5}	m^2 effective leak area
`fouling`	0.35	fraction of flow area lost
`pump_wear`	0.18	fraction of head lost
`stiction`	0.10	deadband, fraction of stroke (slip default 0.04)
`sensor_bias`	25	tag units
`sensor_drift`	1.5	tag units per minute
`sensor_stuck`	none	
`sensor_noise`	2.0	σ in tag units

Sensor faults also take a tag (LT101, LT102, FT101, FT102, PT101 or ZT101) and every fault takes a ramp in seconds.

14.3 Command-line options

14.3.1 Workbench

```
python -m pyplant.workbench.app [--workspace ./work/demo] [--base-port 5400]
                                [--host 127.0.0.1] [--port 8050] [--debug]
```

14.3.2 Services

These are the processes the supervisor starts; the **Runtime** tab shows the exact command line of each. They can be run by hand in four terminals.

Flag	Default	Meaning
`python -m pyplant.services.plant`		
`-port`	5020	Modbus slave port
`-control-port`	0	fault injection HTTP API; 0 disables
`-dt`	0.05	integration step, s
`-speed`	1.0	real-time factor
`-start`	steady	`steady` or `empty` initial state
`-no-noise`	off	disable instrument noise
`python -m pyplant.services.plc`		
`-plant-host`, `-plant-port`	127.0.0.1, 5020	where the plant is
`-port`	5021	PLC Modbus server port
`-opcua-port`, `-no-opcua`	4840	OPC UA server
`-period`	0.10	scan period, s
`-speed`	1.0	real-time factor
`-setpoint`	300	initial setpoint, mm
`python -m pyplant.services.historian`		
`-endpoint`		PLC OPC UA endpoint
`-db`	pyplant.duckdb	database file
`-publish-ms`	200	subscription publishing interval
`-flush`	1.0	buffer flush period, s
`-heartbeat`	30	maximum gap per tag, s
`-api-port`	0	query API port; 0 disables
`python -m pyplant.services.hmi`		
`-endpoint`		PLC OPC UA endpoint
`-history`		historian URL, or a DuckDB path
`-port`, `-poll`	8080, 0.5	HTTP port, poll period (s)

14.3.3 Training from the command line

```
python -m pyplant.ml.train --out artifacts --duration 3600 --magnitudes 3 \
      --repeats 2 --normal-runs 6 --window 60 --stride 10 --quantile 0.995 \
      --seed 0 [--reuse]
```

This fits PCA and the autoencoder; `-reuse` skips generation when `dataset.parquet` already exists in `-out`.

14.4 Code constants

Some parameters are design decisions and live in code. Change them with care and run `pytest` afterwards; the tests check most invariants.

Constant	File	Meaning
`Params` fields	`core/hybrid_tank.py`	plant geometry, pump, valve, orifices (Table 6.2)
`PidTuning` fields	`plc/pid.py`	K_c, T_i, T_d, T_f, limits, action
alarm limits	`tags.py`	`limits` on each tag: HI 500, HIHI 550, LOLO 50 mm; PT HIHI 58 kPa; FT LO 1 L/min
deadbands	`plc/program.py`	10 mm, 2 kPa, 0.5 L/min
`MIN_FLOW_DELAY`	`plc/program.py`	15 s low-flow trip delay
`DEFAULT_NOISE`	`faults/scenarios.py`	instrument noise σ per tag
`NORMAL_MOVES`	`faults/scenarios.py`	setpoint programme of the scenarios
`WINDOW`, `STRIDE`	`ml/features.py`	default window and stride
`PERSISTENCE`	`ml/evaluate.py`	consecutive windows for a detection (3)
`DEVELOPED_AFTER`	`ml/evaluate.py`	settling time before isolation is judged (300 s)
`DATA_PRESETS`, `PRESETS`	`workbench/app.py`	dataset and fault presets in the UI

14.5 Offline experiments with other parameters

The fastest way to try a different plant or tuning is the offline harness: the same PLC program, the same physics, no network, about 1900 times real time.

```
from pyplant.core.hybrid_tank import Inputs, Params
from pyplant.plc.harness import OperatorPanel, PlcHarness, scheduled_setpoint
from pyplant.plc.pid import PidTuning
from pyplant.plc.program import Program
from pyplant.runners.base import Simulation
from pyplant.runners.fast import run

plant = Params(area_tank1=0.03) # a wider tank 1
tuning = PidTuning(kc=0.6, ti=50.0) # a more aggressive loop
harness = PlcHarness(program=Program(ts=0.1, tuning=tuning),
                     panel=OperatorPanel(setpoint=300.0))
sim = Simulation.at_steady_state(Inputs(0.662, True), params=plant,
                                 dt=0.05, control_period=0.1, policy=harness)
sim.policy = scheduled_setpoint(harness, [(0.0, 300.0), (60.0, 350.0)])
tr = run(sim, duration=600.0, log_period=1.0, extra=harness.tags)
df = tr.to_dataframe() # LT101, LIC101_OP, ... one row per second
```

To generate scenario data with a different plant, pass `params` to the scenario: `Scenario("wide", params=Params(area_tank1=0.03), ...)`. Note that `run_scenario()` always uses the default `PidTuning`.

14.5.1 Retuning after a plant change

If you change the plant, re-identify K and τ from an open-loop step (Chapter 5) and recompute the IMC settings:

$$K_c = \frac{\tau}{K\lambda}, \qquad T_i = \tau, \qquad \lambda \approx \tau/5.$$

A larger tank area increases τ roughly in proportion; the gain K changes with the operating point because the valve and the drains are non-linear.

Chapter 15

Extending the toolbox

pyplant is meant to be changed. This chapter gives recipes for the four most common extensions: a new tag, a new fault, a new detector and an online inference service. In every case the rule is the same: change the single source of truth, then let the tests tell you what else must follow.

15.1 Adding a tag

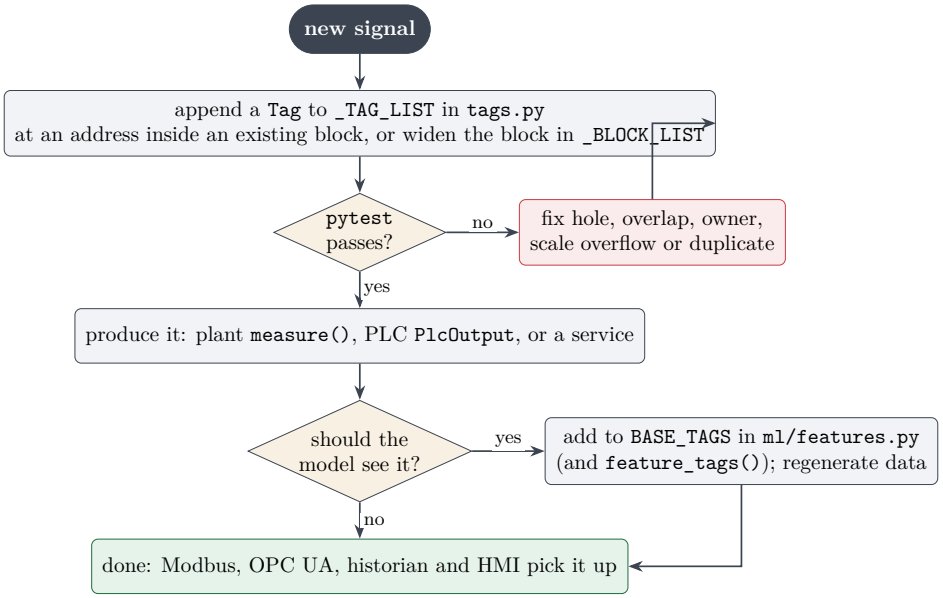


Figure 15.1: Adding a tag.

Because every layer derives its I/O from the tag dictionary, a new tag appears in the Modbus map, the OPC UA address space (with its unit and range), the historian and the HMI's tag list without further code.

DIAG_ORDER is append-only

TOP_CONTRIB stores the top channel as an index into DIAG_ORDER. Reordering it changes the meaning of every historised value. Only append.

15.2 Adding a fault

1. Decide the entry point (Chapter 10). Either subclass `ParameterFault` and implement `apply()`, or subclass `SignalFault` and implement `corrupt()`. A new *command* fault also

needs a branch in `FaultSet.command()`, which today dispatches only `ValveStiction`.

2. Set `kind` and `target` in `__post_init__` and return the magnitude from the `magnitude` property.
3. Register it in `FAULT_KINDS` with the attribute a single magnitude number sets. The injection API and the Faults tab dropdown read this table.
4. Add a starting magnitude and unit to `DEFAULT_MAGNITUDE` and `MAGNITUDE_UNITS` in `workbench/app.py`.
5. Add a representative scenario to `library()` so it enters the dataset grid, and, if its magnitude attribute has a new name, teach `_scaled()` to scale it.
6. If physics names the channel it should isolate to, add it to `EXPECTED_CHANNEL` in `ml/evaluate.py`.
7. Write a test that the fault does what it claims, in `tests/test_faults.py`.

```
@dataclass
class OutletBlockage(ParameterFault):
    """Partial blockage of the tank 2 drain, as fraction of area lost."""
    loss: float = 0.3

    def __post_init__(self) -> None:
        super().__post_init__()
        self.kind, self.target = "outlet_blockage", "FE-2OUT"

    @property
    def magnitude(self) -> float:
        return self.loss

    def apply(self, p: Params, s: float) -> Params:
        return replace(p, cda_2out=p.cda_2out * (1.0 - self.loss * s))

FAULT_KINDS["outlet_blockage"] = (OutletBlockage, "loss")
```

15.3 Adding a detector

All detectors share the `Detector` base class, which provides standardisation, calibration, health index, contributions and persistence. A new method needs only two hooks:

`_fit_core(Z)` fit on standardised normal windows.

`_reconstruct(Z)` return the reconstruction $\hat{Z}$. The score is then $\|Z - \hat{Z}\|^2$ and contributions come for free. A method with no reconstruction returns zeros and overrides `score()` instead, as the isolation forest does.

```
from sklearn.covariance import EmpiricalCovariance

@dataclass
class MahalanobisDetector(Detector):
    """Hotelling-style distance using the full normal covariance."""
    cov: EmpiricalCovariance | None = None

    def __post_init__(self) -> None:
        self.kind = "mahalanobis"
```

```
    def _fit_core(self, Z):
        self.cov = EmpiricalCovariance().fit(Z)

    def _reconstruct(self, Z):
        return np.zeros_like(Z) # contributions fall back to z^2

    def score(self, X):
        return self.cov.mahalanobis(self.scaler.transform(X))
```

Then add it to `METHODS` (label shown in the Model tab, in order from simplest to most complex) and a branch in `make_detector()`. The Model tab checklist and the selection rule pick it up automatically. Keep scores positive with larger meaning worse, or the health index breaks.

15.4 Adding an online inference service

The `ML_RESULT` and `ML_ALARMS` blocks are reserved for a service that scores the live plant. A minimal design:

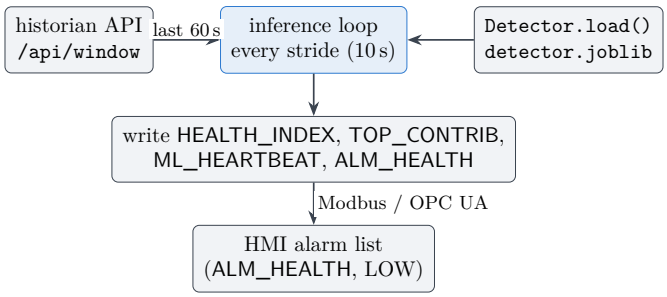


Figure 15.2: Sketch of an inference service.

1. Every stride, request the last `window` seconds of `BASE_TAGS` from the historian at 1 Hz.
2. Apply `add_derived()` and `windows()` to get one feature row.
3. Compute `health()` and `top_channel()`; map the channel to its `DIAG_ORDER` index (0 for derived channels or no dominant tag).
4. Write the results to the `ML_RESULT` block on the PLC's Modbus server; increment ML_HEARTBEAT so a stalled service is visible.
5. Set ALM_HEALTH after the same persistence rule used in evaluation (three consecutive windows above 50).
6. Add it to `SERVICES` in `workbench/supervisor.py` after the historian.

Remember that historian timestamps are wall clock: at `speed` > 1, ask for `window / speed` wall seconds.

15.5 Testing

Run the whole suite from the project root:

```
python -m pytest -q # about 370 tests
python -m pytest tests/test_ml.py -q -k detector
```

The tests encode the invariants this manual describes: codec round trips, registry rules, conservation of mass, step independence, PID properties, drift-free scheduling, alarm state transitions, write guards, stiction cycles, split leakage and the detector interface. A change that breaks one of them has usually broken an assumption somewhere else too.

Chapter 16

API reference

Every service has a small JSON-over-HTTP API, and every layer has a Python API. This chapter lists both. Ports are the workbench defaults with base 5400 (Table 2.1); add your own base offset if you changed it.

16.1 Plant control API (port 5401)

Fault injection for the live plant. It is deliberately separate from anything the control system can see: an operator, the HMI or a diagnosis model must not be able to find out that a fault was injected except by observing the plant.

Method	Path	Purpose
GET	`/api/faults`	plant time, available kinds, real faults, instrument noise entries, ids active now
POST	`/api/fault`	inject a fault; returns its label; 422 if invalid
DELETE	`/api/fault/{id}`	remove one fault; 404 if unknown
DELETE	`/api/faults`	remove all real faults, keep instrument noise
GET	`/api/truth`	current label, true and reported measurements

The `POST /api/fault` body:

Field	Default	Meaning
`kind`	required	one of `FAULT_KINDS`
`id`	`<kind>_<t>`	unique id
`magnitude`	kind default	value of the kind's magnitude attribute
`tag`	`LT101`	sensor faults only
`delay`	0	plant seconds from now until onset
`duration`	none	plant seconds until the fault ends
`ramp`	0	seconds to full magnitude

```
# PowerShell
Invoke-RestMethod -Method Post -Uri http://127.0.0.1:5401/api/fault ‘
  -ContentType application/json ‘
  -Body ’{"kind":"leak","magnitude":5e-5,"ramp":300}’
Invoke-RestMethod http://127.0.0.1:5401/api/truth

# bash / curl
curl -X POST localhost:5401/api/fault -H "Content-Type: application/json" \
     -d ’{"kind":"sensor_bias","tag":"LT101","magnitude":25}’
curl -X DELETE localhost:5401/api/faults
```

16.2 Historian query API (port 5404)

Method	Path	Purpose
GET	`/api/health`	row count, tag count, time span, collector statistics
GET	`/api/window`	query `tags=A,B`, `seconds=300`, `freq=1`: the last `seconds` of the listed tags on a regular grid (LOCF); `null` where no value yet; 404 for unknown tags
GET	`/api/latest`	query `tags=A,B`: most recent good value per tag (all tags if omitted)

16.3 HMI API (port 5405)

Method	Path	Purpose
GET	`/`	the operator page
GET	`/api/state`	connected flag, last update, all values, active alarm list, alarm summary, poll and error counts
GET	`/api/tags`	tag dictionary: description, unit, span, kind, writable
GET	`/api/trend`	query `tags`, `seconds`, `freq`: trend data via the historian; 503 if none
POST	`/api/setpoint`	body `{"value": 320}`: write LIC101_SP
POST	`/api/manual`	body `{"value": 70}`: write LIC101_MAN_OP
POST	`/api/mode`	body `{"auto": false}`: auto or manual
POST	`/api/pump`	body `{"start": true}`: pump start request
POST	`/api/reset`	pulse alarm and interlock reset
POST	`/api/alarms/ack`	body `{"tag": "ILK_PMP101"}`: acknowledge one alarm; `null` acknowledges all
POST	`/api/alarms/shelve`	body `{"tag": ..., "minutes": 30, "reason": ...}`; at most 8 h
POST	`/api/alarms/unshelve`	body `{"tag": ...}`: return a shelved alarm to service

Writes are guarded (404 unknown tag, 403 not writable, 422 out of span, 503 not connected), as described in Chapter 9.

16.4 Industrial protocols

Modbus TCP, plant (5400) holding registers 0–5 (measurements), coil 0 (pump running); the PLC writes HR 10 (valve) and coil 10 (pump).

Modbus TCP, PLC (5402) the full tag map of Appendix A.1. Any Modbus client can read it; the operator blocks are writable.

OPC UA, PLC (5403) endpoint
`opc.tcp://127.0.0.1:5403/pyplant/`,
nodes under `Plant/Unit1`, `PLC/LIC101`, `PLC/Commands`, `PLC/Alarms`, `PLC/Diagnostics`, with engineering units and ranges. Browse it with any OPC UA client, e.g. UaExpert.

16.5 Python API

16.5.1 Tags

```
from pyplant.tags import tag, tags_in, TAGS, BLOCKS
lt = tag("LT101")
lt.encode(432.7) # 4327 on the wire
lt.decode(4327) # 432.7
lt.path # 'Plant/Unit1/LT101'
lt.limits.hihi # 550.0
tags_in("PLANT_MEAS") # the six tags polled in one read
```

16.5.2 Plant physics

```
import numpy as np
from pyplant.core.hybrid_tank import Params, Inputs, step, steady_state, measure
p, u = Params(), Inputs(valve_cmd=0.662, pump_on=True)
x = steady_state(u, p) # analytic equilibrium
x = step(x, u, p, dt=0.05) # one RK4 step
measure(x, u, p) # {'LT101': 300.3, 'FT101': 10.05, ...}
```

16.5.3 Simulation and runners

```
from pyplant.runners.base import Simulation, piecewise
from pyplant.runners.fast import run, settle
sim = Simulation.at_steady_state(Inputs(0.60, True), policy=piecewise([
    (0.0, Inputs(0.60, True)), (300.0, Inputs(0.70, True))]))
tr = run(sim, duration=1800.0, log_period=1.0)
tr.to_dataframe()
```

16.5.4 Main modules

Object	Purpose
`plc.pid.PID`, `PidTuning`	velocity PID; `step()`, `track()`
`plc.program.Program`	one scan: `execute(PlcInput) -> PlcOutput`
`plc.harness.PlcHarness`	program as a simulation policy, no network
`plc.scan.ScanEngine`	real-time scan cycle over an `IOLink`
`clock.Ticker`	drift-free periodic wake-ups
`faults.model`	fault classes, `FaultSet`, `build_fault`
`faults.scenarios`	`Scenario`, `run_scenario`, `library`, `grid`
`hist.db.Historian`	DuckDB store; `write`, `get_window`, `covered_span`
`hmi.alarms.AlarmManager`	ISA-18.2 alarm list
`ml.dataset`	`build`, `default_dataset`, `split_scenarios`, `windowed`
`ml.features`	`add_derived`, `windows`, `WindowSpec`
`ml.detector`	detectors, `make_detector`, `METHODS`
`ml.evaluate.evaluate`	`Report` with FAR, delay, isolation
`workbench.workspace.Workspace`	config, run log, paths
`workbench.supervisor.Supervisor`	start and stop the four services

Chapter 17

Troubleshooting

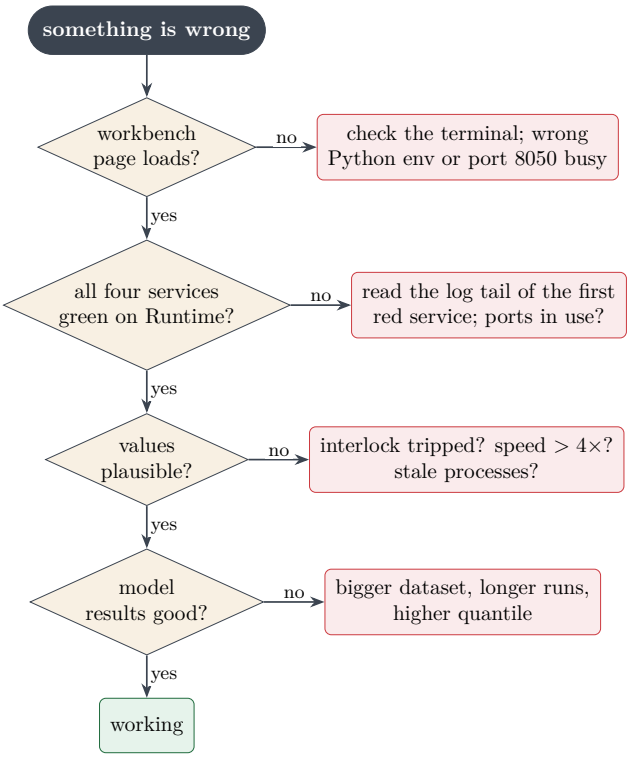


Figure 17.1: Where to look first.

17.1 Installation and start-up

`SyntaxError` or `ImportError`: `StrEnum` at start. The project needs Python 3.11 or later: the alarm module uses `enum.StrEnum`, although `pyproject.toml` still says 3.10. The base interpreter on many machines is older; activate the project environment first (`conda activate pyplant`) or call its `python.exe` directly.

`pymodbus` errors such as unexpected keyword `slave`. The Modbus layer is written against pymodbus 3.8. Reinstall the pinned version: `pip install "pymodbus>=3.8,<3.9"`.

"ports already in use by something else". Another stack, or leftover processes from a crashed session, hold the ports. Start with a different base (`-base-port 5500`) or stop the leftovers. On Windows:

```
Get-CimInstance Win32_Process |
  Where-Object { $_.CommandLine -match 'pyplant\.(services|workbench)' } |
  ForEach-Object { Stop-Process -Id $_.ProcessId -Force }
```

On Linux or macOS: `pkill -f pyplant.services`.

A service turns red right after start. The Runtime tab shows the last lines of its log, and the full log is in `<workspace>/jobs/<name>.log`. Services start in order (plant, PLC, historian, HMI), so fix the first red one.

17.2 The live stack

The page is slow or callbacks lag. When a service is down, each request to it would wait for the connection to be refused, which takes about 2 s on Windows. The workbench connects with a 0.5 s timeout and then reports the service as down for 2.5 s without retrying. If the page is still slow, stop the stack cleanly and restart it.

SCAN_OVERRUNS keeps rising, the loop is sluggish. The speed is too high for the operating system's timer. Use $\leq 4\times$ on Windows.

The level drops to zero and the pump is off. The interlock has tripped, usually on low flow after the valve was driven shut (manual output 0%) or after a large leak. Remove the cause, then press **Reset** on the HMI. Acknowledging the alarm alone does not reset the trip.

The HMI says "not connected". The HMI talks to the PLC over OPC UA. Check that the PLC service is green and that port 5403 is listening; the HMI reconnects on its own once the PLC is up.

Trends are empty for the first seconds. The historian has to receive a value of every requested tag before a window is complete. Wait a few seconds after start, or longer for quiet tags (heartbeat 30 s).

I injected a fault and nothing happens. Often correct: integral action holds the level on setpoint (Section 7.5). Watch the controller output and `GET /api/truth`. Slow faults (fouling, wear with ramps, stiction) need several minutes of plant time.

17.3 Data and models

"need at least three normal runs". Raise **Normal runs** on the Data tab to 3 or more (6 is a sensible minimum).

"there is no dataset yet". Generate one on the Data tab before training. Datasets are stored in the workspace and survive restarts.

`PermissionError` on a job state file (Windows). Antivirus or the indexer briefly locks a file the job is rewriting. The job runner retries for up to 0.2 s; if it persists, exclude the workspace directory from real-time scanning.

Detection rate is low for one fault mode. Check the per-mode table: if the fault ramps for 20 minutes and runs are 20 minutes long, it never develops. Use 3600 s runs. Low-magnitude variants in the grid are meant to be hard.

False alarms on normal runs. Increase the quantile (0.999), add normal runs, or increase the window to smooth noise.

The autoencoder is worse than PCA. With little data this is expected and the selection rule will deploy PCA. Use the Standard or Thorough preset.

17.4 Starting over

Deleting the workspace directory is always safe; the next start recreates it with defaults. From Python, `Workspace(path).reset()` deletes data, artefacts, history and the run log but keeps the configuration.

Appendix A

Reference

A.1 Tag list

HR = holding register, C = coil. Scale is the integer factor on the wire.

Tag	Description	Owner	Table	Addr	Unit	Span	Scale
Plant measurements (block `PLANT_MEAS`, `PLANT_STATUS`)							
LT101	tank 1 level	PLANT	HR	0	mm	0–600	10
LT102	tank 2 level	PLANT	HR	1	mm	0–600	10
FT101	pump discharge flow	PLANT	HR	2	L/min	0–20	100
FT102	tank 2 outlet flow	PLANT	HR	3	L/min	0–20	100
PT101	pump discharge pressure	PLANT	HR	4	kPa	0–100	100
ZT101	valve stem position	PLANT	HR	5	%	0–100	100
PMP101_RUN	pump running	PLANT	C	0			
Actuator commands (`PLC_CMD`, `PLC_CMD_BITS`)							
FCV101_OP	valve command	PLC	HR	10	%	0–100	100
PMP101_CMD	pump start command	PLC	C	10			
Operator (`OPER_SP`, `OPER_CMD`); writable							
LIC101_SP	setpoint	OPER	HR	20	mm	0–600	10
LIC101_MAN_OP	manual output demand	OPER	HR	21	%	0–100	100
LIC101_AUTO	loop in automatic	OPER	C	11			
CMD_ALM_RESET	alarm and interlock reset	OPER	C	12			
CMD_PMP101_START	pump start request	OPER	C	13			
Faceplate readback (`PLC_FACE`)							
LIC101_PV	process value	PLC	HR	22	mm	0–600	10
LIC101_OP	controller output	PLC	HR	23	%	0–100	100
LIC101_ERR	control error	PLC	HR	24	mm	±600	10
Diagnostics (`PLC_DIAG`)							
SCAN_MS	scan time	PLC	HR	30	ms	0–1000	10
SCAN_OVERRUNS	overrun count	PLC	HR	31	count	0–30000	1
PLC_HEARTBEAT	heartbeat counter	PLC	HR	32	count	0–30000	1
Alarms and interlocks (`PLC_ALARMS`)							
ALM_LT101_HI	tank 1 level high	PLC	C	20			
ALM_LT101_LOLO	tank 1 level low low	PLC	C	21			
ALM_PT101_HIHI	pressure high high	PLC	C	22			
ILK_PMP101	pump interlock latched	PLC	C	23			
Inference results (`ML_RESULT`, `ML_ALARMS`); reserved							
HEALTH_INDEX	model health index	ML	HR	40	%	0–100	100
TOP_CONTRIB	top contributing tag	ML	HR	41	enum	0–64	1
ML_HEARTBEAT	inference heartbeat	ML	HR	42	count	0–30000	1
ALM_HEALTH	health index degraded	ML	C	30			

Print the live list and block plan with `python -m pyplant.tags`.

A.2 File layout

```
pyplant/
  tags.py tag registry, codec, block plan
  clock.py drift-free Ticker
  core/hybrid_tank.py plant physics (pure functions, SI units)
  runners/base.py Simulation stepper, zero-order hold, policies
  runners/fast.py non-real-time driver, Trajectory, settle
  runners/realtime.py real-time plant task
  plc/pid.py velocity-form PID
  plc/program.py image tables, alarms, interlocks, timers
  plc/harness.py in-process PLC wiring for datasets and experiments
  plc/scan.py scan engine, IOLink, diagnostics
  net/store.py, link.py Modbus datastore, TagServer, TagClient
  net/opcua.py OPC UA address space, server view, client
  services/plant.py plant node (Modbus slave + fault API)
  services/plc.py soft PLC node (Modbus client/server + OPC UA)
  services/historian.py OPC UA collector + query API
  services/hmi.py HMI process
  hist/db.py DuckDB store, LOCF windows
  hmi/alarms.py ISA-18.2 alarm state machine
  hmi/app.py, page.html HMI API and operator page
  faults/model.py fault objects, FaultSet, FAULT_KINDS
  faults/scenarios.py scenario library, grid, run_scenario
  ml/features.py windows, statistics, derived channels
  ml/dataset.py generation, Parquet, scenario-level split
  ml/detector.py z-score, PCA, isolation forest, autoencoder
  ml/evaluate.py false alarms, delay, isolation
  ml/train.py command-line training pipeline
  workbench/app.py Dash workbench
  workbench/workspace.py configuration, run log, paths
  workbench/supervisor.py start and stop the services
  workbench/jobs.py background jobs
  workbench/tasks.py dataset, training and quick-start tasks
  workbench/clients.py HTTP clients for the services
tests/ pytest suite
manual/ this manual
```

A.3 Glossary

Anti-windup Preventing the integral term from accumulating while the actuator is saturated.

ASOF join A join that matches each row to the latest row of another table at or before its time; used for LOCF resampling.

Autoencoder A neural network trained to reproduce its input through a narrow bottleneck.

Bumpless transfer Switching between manual and auto without a jump in controller output.

Contribution A channel's share of a window's anomaly score.

EURange OPC UA property giving a variable's engineering range.

FAR False alarm rate, here persistent alarms per day on normal data.

Health index Anomaly score mapped to 0–100 with the threshold at 50.

HMI Human–machine interface, the operator's screen.

Hysteresis Different set and clear thresholds to stop chattering.

IMC Internal model control, a model-based tuning method.

Image table A PLC's snapshot of inputs or outputs for one scan.

Interlock Protective logic that forces a safe state and latches.

ISA-18.2 The standard for alarm management in process industries.

Isolation forest Tree ensemble that scores how easily a point is isolated by random splits.

LOCF Last observation carried forward.

Modbus TCP Simple register-based field protocol.

MSPC Multivariate statistical process control.

OPC UA Information-model protocol with typed, browsable nodes.

OP, PV, SP Controller output, process value, setpoint.

P&ID Piping and instrumentation diagram.

PCA Principal component analysis.

Persistence Consecutive flagged windows required for a detection.

PLC Programmable logic controller.

RK4 Fourth-order Runge–Kutta integration.

SCADA Supervisory control and data acquisition.

Scan cycle Read inputs, execute logic, write outputs, repeat.

Shelving Temporary, expiring suppression of an alarm.

SPE (Q) Squared prediction error off the PCA subspace.

Stiction Static friction in a valve: sticking, then slipping.

TON On-delay timer.

Velocity form PID computing the change in output each scan.

ZOH Zero-order hold; output held constant between samples.